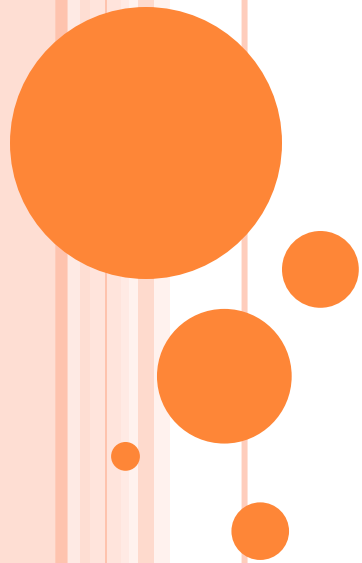


SEQUENTIAL CIRCUITS

3.1_ Flip Flops

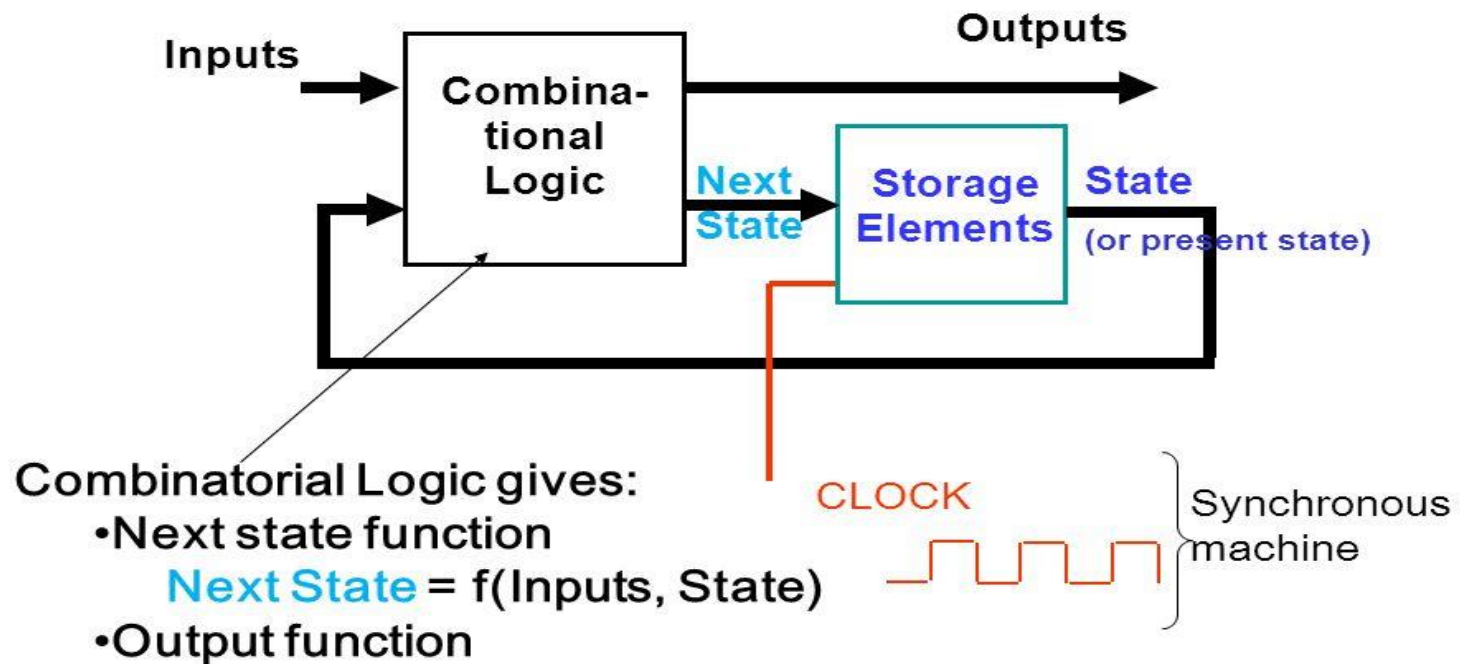


SEQUENTIAL CIRCUITS

- In Sequential logic circuits present output depends on the present input as well as past output / outputs
- Sequential circuit can be considered as combinational circuit with feedback circuit. Sequential circuit uses a memory element like flip – flops as feedback circuit in order to store past values.



BLOCK DIAGRAM OF SEQUENTIAL CIRCUIT



Combinational Circuits vs Sequential Circuits

COMBINATIONAL CIRCUITS

Output depends only on the present value of the inputs.

These circuits will not have any memory as their outputs change with the change in the input value.

There are no feedbacks involved.

Used in basic Boolean operations.

Implemented in: Half adder circuit, full adder circuit, multiplexers, demultiplexers, decoders and encoders.



SEQUENTIAL CIRCUITS

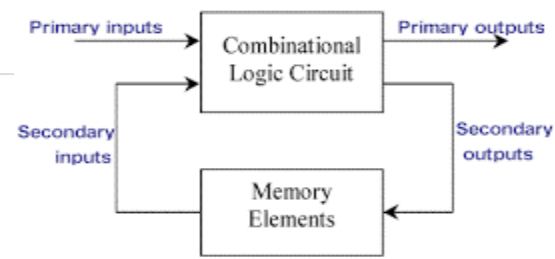
Output depends on both the present and previous state values of the inputs

Sequential circuits have some sort of memory as their output changes according to the previous and present values.

In a sequential circuit the outputs are connected to it as a feedback path.

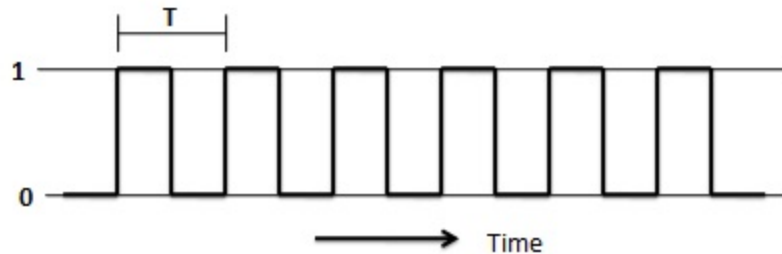
Used in the designing of memory devices.

Implemented in: RAM, Registers, counters and other state retaining machines.

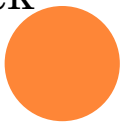


CLOCK SIGNAL IN SEQUENTIAL CIRCUITS

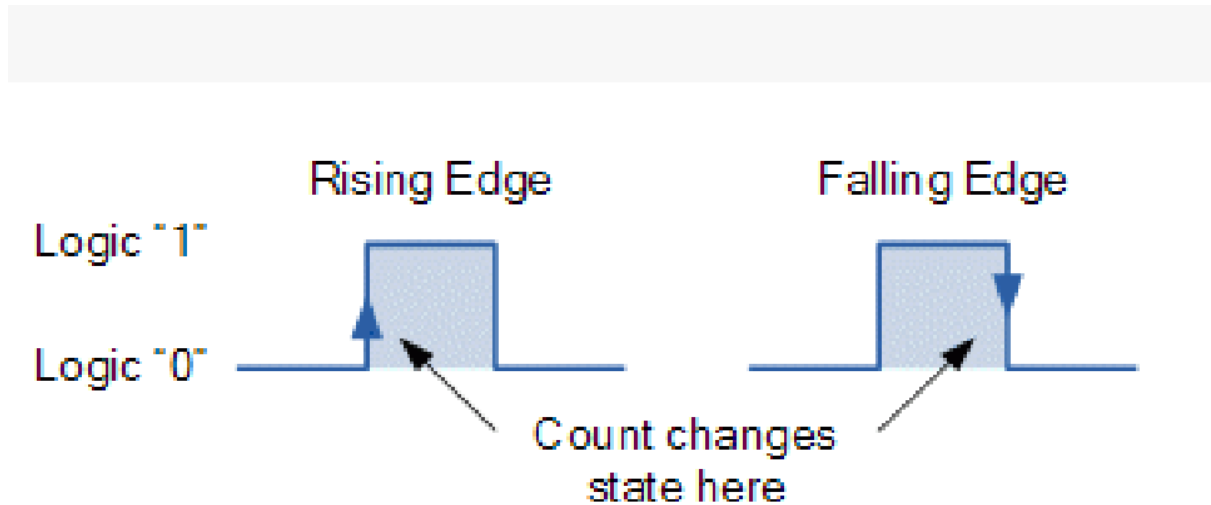
- The clock signal plays a crucial role in sequential circuits. A clock is a signal, which oscillates between logic level 0 and logic level 1, repeatedly. Square wave with constant frequency is the most common form of clock signal. A clock signal has “edges”. These are the instants at which the clock changes from 0 to 1 (a positive edge) or from 1 to 0 (a negative edge).



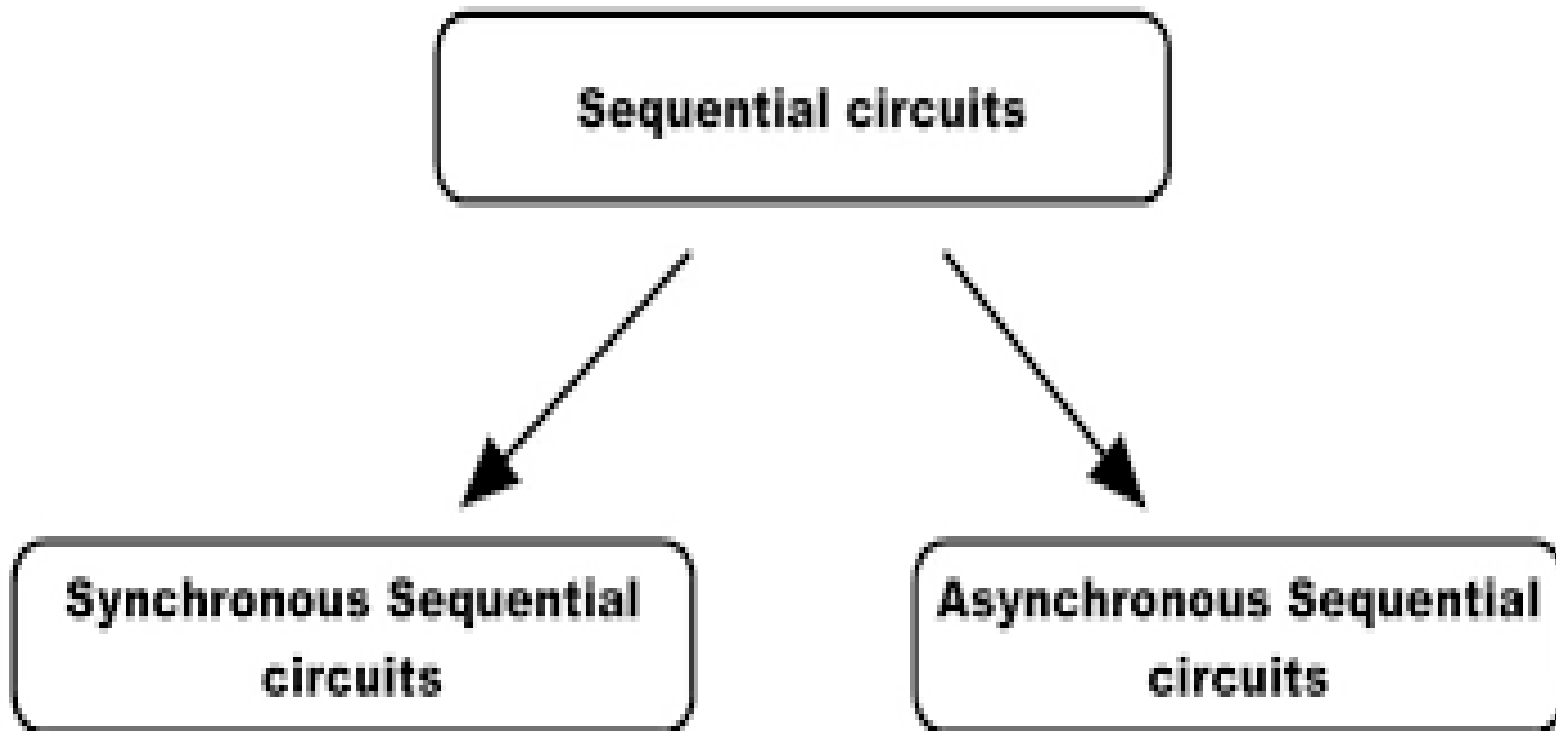
- Clock signals control the outputs of the sequential circuit. That is it determines when and how the memory elements change their outputs. If a sequential circuit is not having any clock signal as input, the output of the circuit will change randomly. So that they cannot retain their state till the next input signal arrives. But sequential circuits with clock input will retain its state till the next clock edge occurs.



CLOCK SIGNAL FOR EDGE TRIGGERING



SEQUENTIAL CIRCUITS CLASSIFICATION

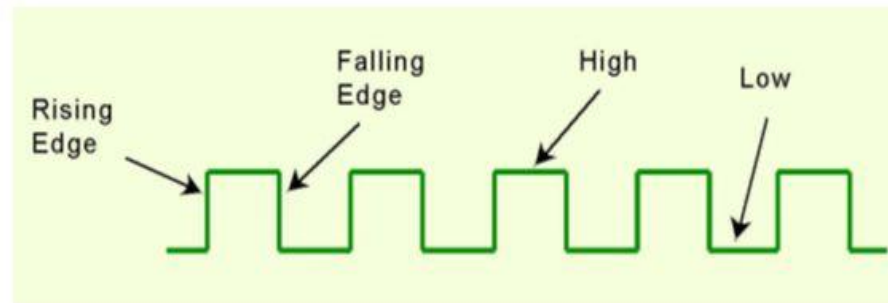


Sequential Logic Circuits



Types of Sequential Logic Circuits:

- A **synchronous** sequential circuit uses a clock to order events
 - a clock is a circuit that emits a series of electrical pulses
 - state changes in sequential circuits only occur when the clock ticks



- **Asynchronous** sequential circuits become active the moment any input value changes



LATCHES

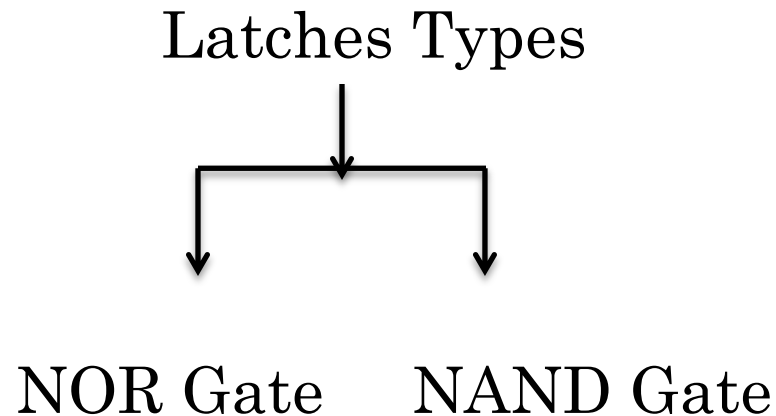
- Latch is a basic building element in sequential circuits. Latches do not have any clock signal , that is they are asynchronous sequential circuits.
- Latches are basic storage elements that operate with signal levels (rather than signal transitions).
- Latches controlled by a clock transition are flip-flops. Latches are level-sensitive devices.
- Latches are useful for the design of the asynchronous sequential circuit.



- Latches are made up of static gates.
- Latch is a bistable multivibrator i.e. it has two stable states and can switch between these states.
- Latches will have a feedback path from the output.
- Thus they change their output at any instant using the previous and present states of the input signals.

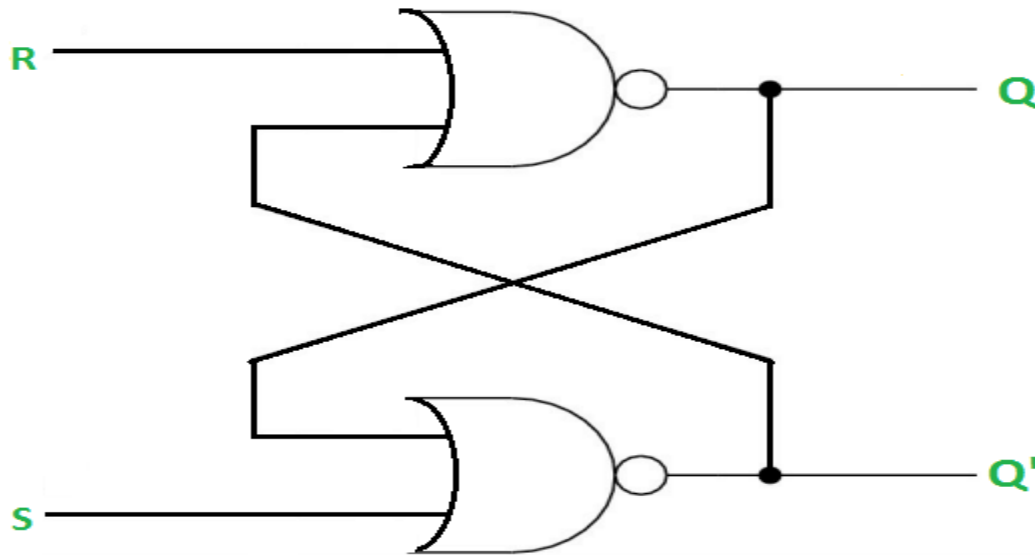


- When enabled, the output of the latch is continuously affected by its input i.e. the output changes immediately when the input changes. When disabled, the state of the latch remains constant i.e. it remembers its previous value.

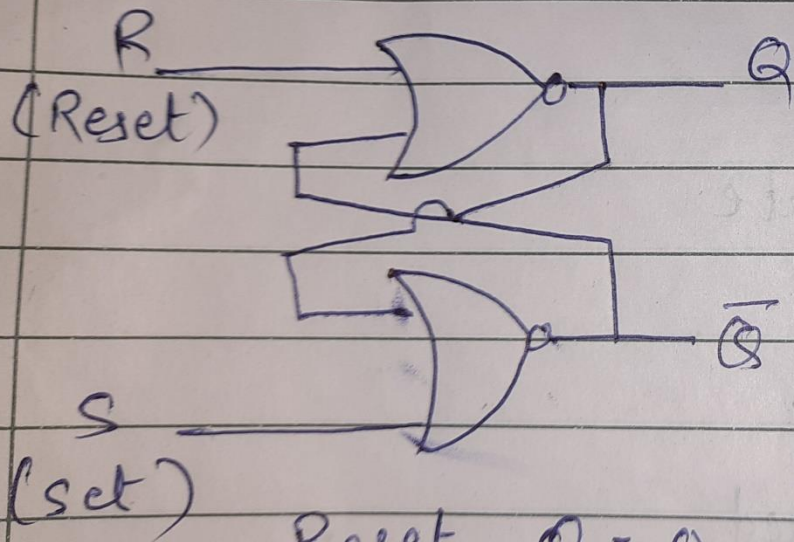


THE SR LATCH USING NOR GATE

- It is an active high S-R latch. That means it is SET when $S = 1$.



SR latch using NOR



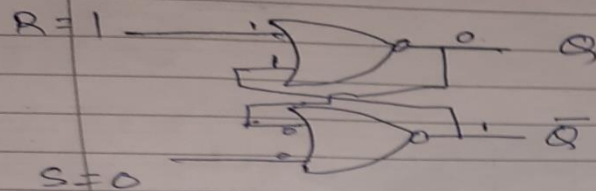
NOR T.T.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Reset $Q = 0$

Set $Q = 1$

Case-1 $S=0, R=1, Q=0, \bar{Q}=1$



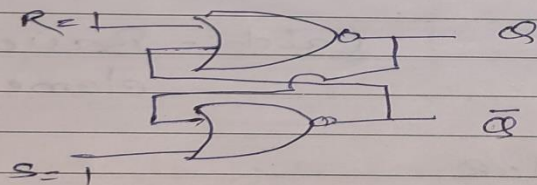
$S=0, R=0, Q=0, \bar{Q}=1$ - memory/last state

Case 2 $S=1, R=0, Q=1, \bar{Q}=0$



if no i/p $S=0, R=0, Q=1, \bar{Q}=0$ - last o/p

Case 3 $S=1, R=1, Q=0, \bar{Q}=0$ - invalid.



if $S=0, R=0$

$Q=0, \bar{Q}=1$

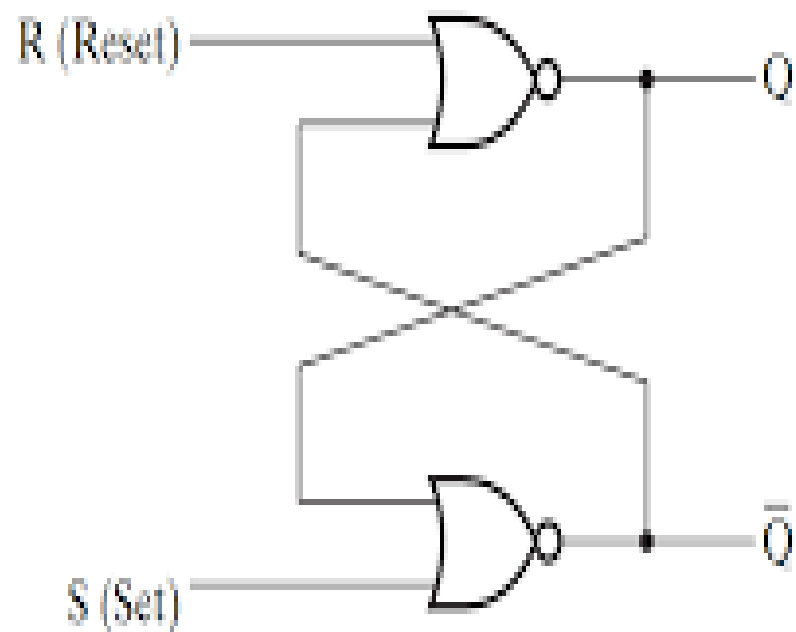
not working as last q

S	R	Q	$\bar{Q}$
0	0	last state	
0	1	0	1
1	0	1	0
1	1	Not used	

WORKING OF SR LATCH (USING NOR)

- NOR gate always gives output 0 when at least one of the inputs is 1.
- So when S is applied as 1 the output of gate G2 i.e. is 0 irrespective of the condition of second input Q to the gate.
- Now is the input of gate G1 so both the inputs of G1 become 0 as R is already 0. So, the output of G1 is now or 1.
- So whatever may be the previous condition of Q, it always becomes $Q = 1$ and $\bar{Q} = 0$ when $S = 1$ and $R = 0$. This is called the SET condition of the latch.
- In the above logic circuit if $S = 0$ and $R = 1$, Q becomes 0. Let us explain how.
- As we already said, a NOR gate always gives output 0 when at least one of the inputs is 1.
- So when R is applied as 1, the output of gate G1 i.e. Q is 0 irrespective of the condition of the second input to the gate.
- So, whatever may be the previous condition of Q, it always becomes 0 this 0 is then fed back to the input of gate G2. As here S is already 0, both inputs of G2 are 0. Hence the output of G2 i.e. will be 1. So, $Q = 0$ and $\bar{Q} = 1$ when, $S = 0$ and $R = 1$. This is called the RESET condition of the latch.





(a) Logic diagram

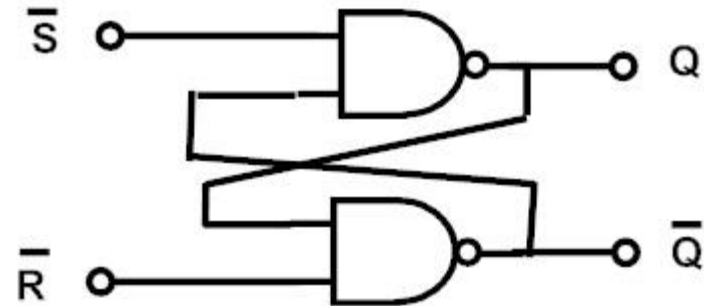
S	R	Q	$\bar{Q}$	
1	0	1	0	Set state
0	0	1	0	
0	1	0	1	Reset state
0	0	0	1	
1	1	0	0	Undefined

(b) Function table



S-R LATCH WITH NAND GATES

- Example: S-R latch is an example for simple latch. Two cross-coupled NAND gates
- S- Set R – Reset

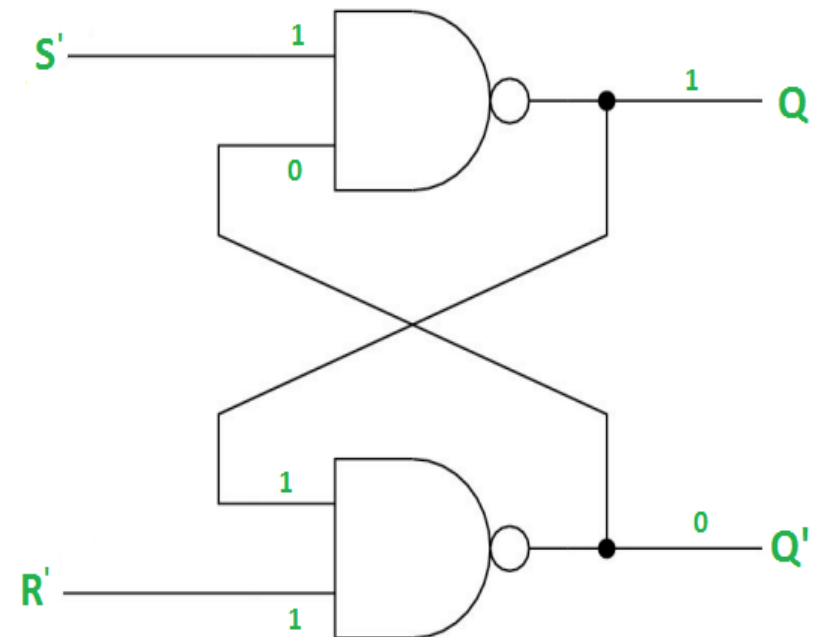
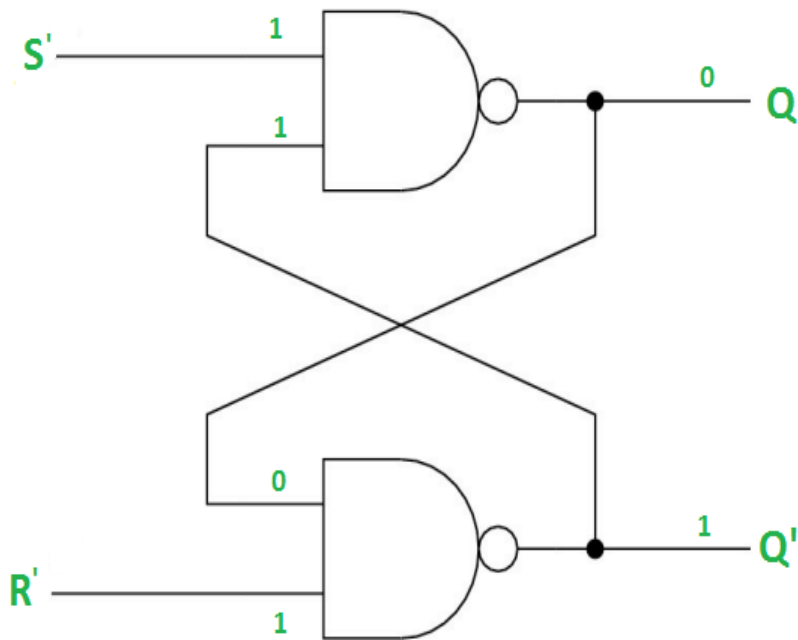


- **Case-1: $S'=R'=1$ ($S=R=0$) –**
If $Q = 1$, Q and R' inputs for 2nd NAND gate are both 1.
If $Q = 0$, Q and R' inputs for 2nd NAND gate are 0 and 1 respectively.



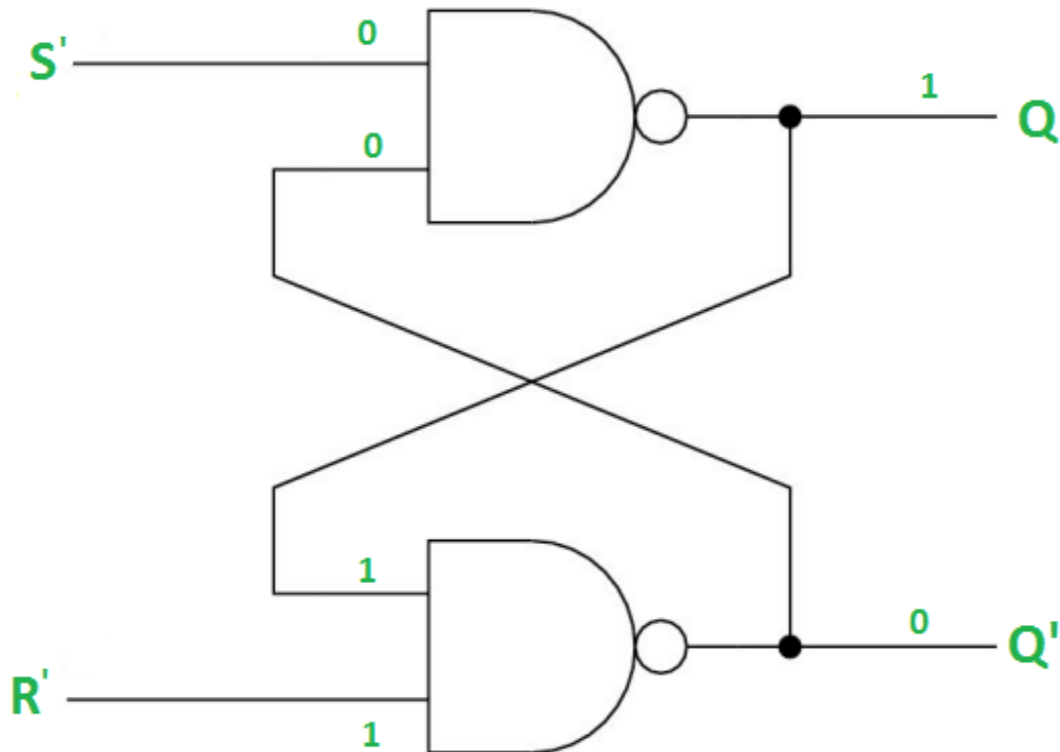
- **Case-2: $S'=0$, $R'=1$ ($S=1$, $R=0$) –**

As $S'=0$, the output of 1st NAND gate, $Q = 1$ (**SET state**). In 2nd NAND gate, as Q and R' inputs are



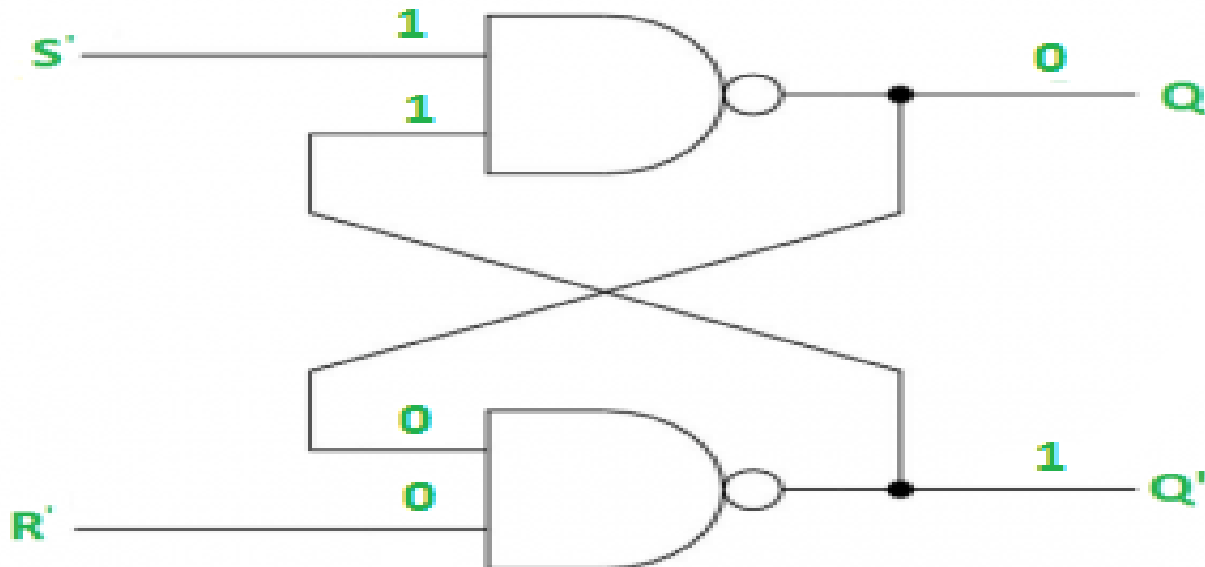
Case-2: $S'=0$, $R'=1$ ($S=1$, $R=0$) –

As $S'=0$, the output of 1st NAND gate, $Q = 1$ (**SET state**). In 2nd NAND gate, as Q and R' inputs are 1, $Q'=0$.



- **Case-3: $S'=1, R'=0$ ($S=0, R=1$) –**

As $R'=0$, the output of 2nd NAND gate, $Q'=1$. In 1st NAND gate, as Q and S' inputs are 1, $Q=0$ (**RESET state**).



TRUTH TABLE OF RS LATCH(NAND GATE)

Case-4: $S'=R'=0$ ($S=R=1$) –

When $S=R=1$, both Q and Q' becomes 1 which is not allowed. So, the input condition is prohibited.

$\bar{S}$	$\bar{R}$	Q	State
1	1	Previous State	No change
1	0	0	Reset
0	1	1	Set
0	0	?	Forbidden



SOME DEFINITIONS RELATED TO SR LATCH

○ Race Condition-

If $S=R=1$, this condition is called as Race condition

When any one input to a NOR gate is 1, its output becomes 0. Thus both outputs try to become 0. This is RACE condition

○ Characteristic Equation-

Characteristic equation of a latch or flip flop is the equation which relates the next state of the flip flop or latch to the current state and inputs.

Characteristic equation is obtained from the truth table of flip flop or latch using K map



Characteristic table for SR latch

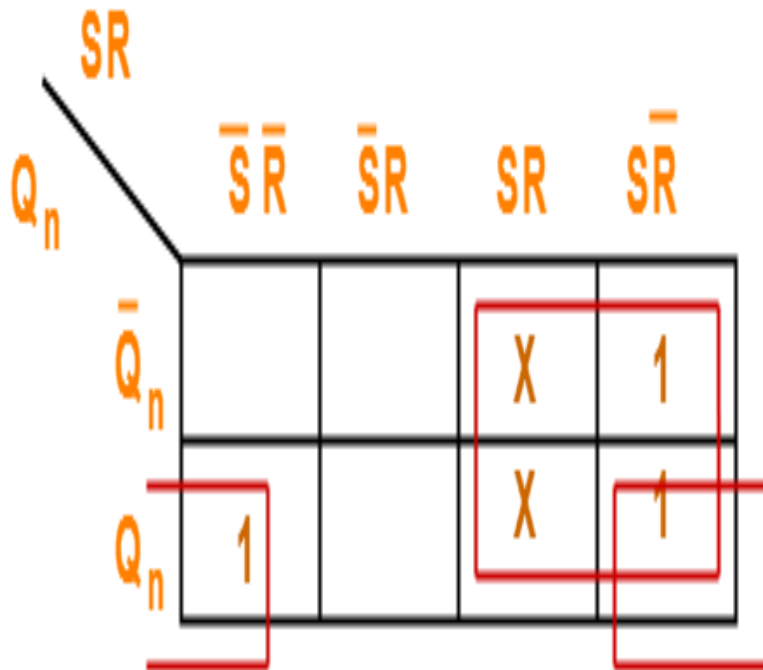
INPUTS			OUTPUTS
S	R	Q_n (Present State)	Q_{n+1} (Next State)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Indeterminate
1	1	1	Indeterminate



Reduced characteristic table of RS Latch

INPUTS			OUTPUTS	REMARKS
S	R	Q_n (Present State)	Q_{n+1} (Next State)	States and Conditions
0	0	X	Q_n	Hold State condition $S = R = 0$
0	1	X	0	Reset state condition $S = 0$, $R = 1$
1	0	X	1	Set state condition $S = 1$, $R = 0$
1	1	X	Indeterminate	Indeterminate state condition $S = R = 1$

CHARACTERISTIC EQUATION OF SR LATCH



A Karnaugh Map for the next state Q_{n+1} of an SR latch. The vertical axis is labeled Q_n with values $\bar{Q}_n$ and Q_n . The horizontal axis is labeled SR with values $\bar{S}\bar{R}$, $\bar{S}R$, SR , and $S\bar{R}$. The map contains 'X' in the $\bar{S}R$ and SR columns for both $\bar{Q}_n$ and Q_n rows, and '1' in the $\bar{S}\bar{R}$ and $S\bar{R}$ columns for both $\bar{Q}_n$ and Q_n rows. Red boxes highlight the 'X' cells and the '1' cells in the $\bar{S}R$ and SR columns. Red lines connect the '1' cells in the $\bar{S}\bar{R}$ and $S\bar{R}$ columns to the output equation.

Q_n	$\bar{S}\bar{R}$	$\bar{S}R$	SR	$S\bar{R}$
$\bar{Q}_n$			X	1
Q_n	1		X	1

K Map

- From the truth table of SR latch- K map for Next state Q_{n+1} is written and simplified.
- Characteristic equation is

$$Q_{n+1} = S + R' Q_n$$



Excitation Table

The excitation table of any latch or flip flop is drawn using its truth table.

What is excitation table?

For a given combination of present state Q_n and next state Q_{n+1} , excitation table tell the inputs required.

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0



FLIP-FLOP

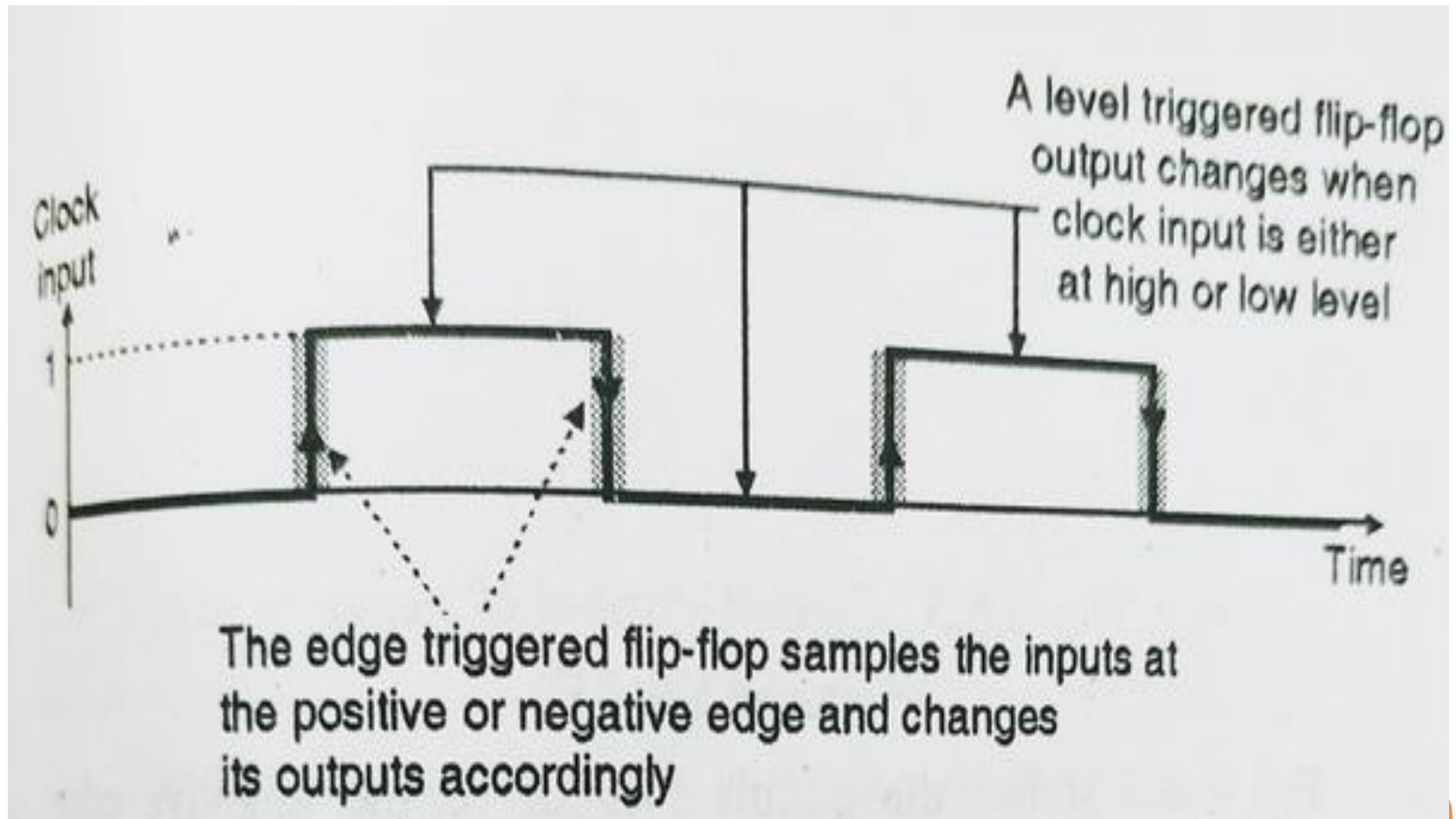
- The Flip-flop is also a building block of synchronous sequential circuits. It has two stable states (Bistable). It can store one bit of information. Flip flops will have a clock signal. Their state changes depending on the clock pulse .These devices have two states and a feedback path .
- Flip-Flop is edge sensitive. They will change their state when the clock signal transition occurs from low to high or high to low.
- After the transition of clock signal from either 0 to 1 or 1 to 0 i.e. when the clock is at constant 0 or 1, the state remains unchanged even if the input changes.



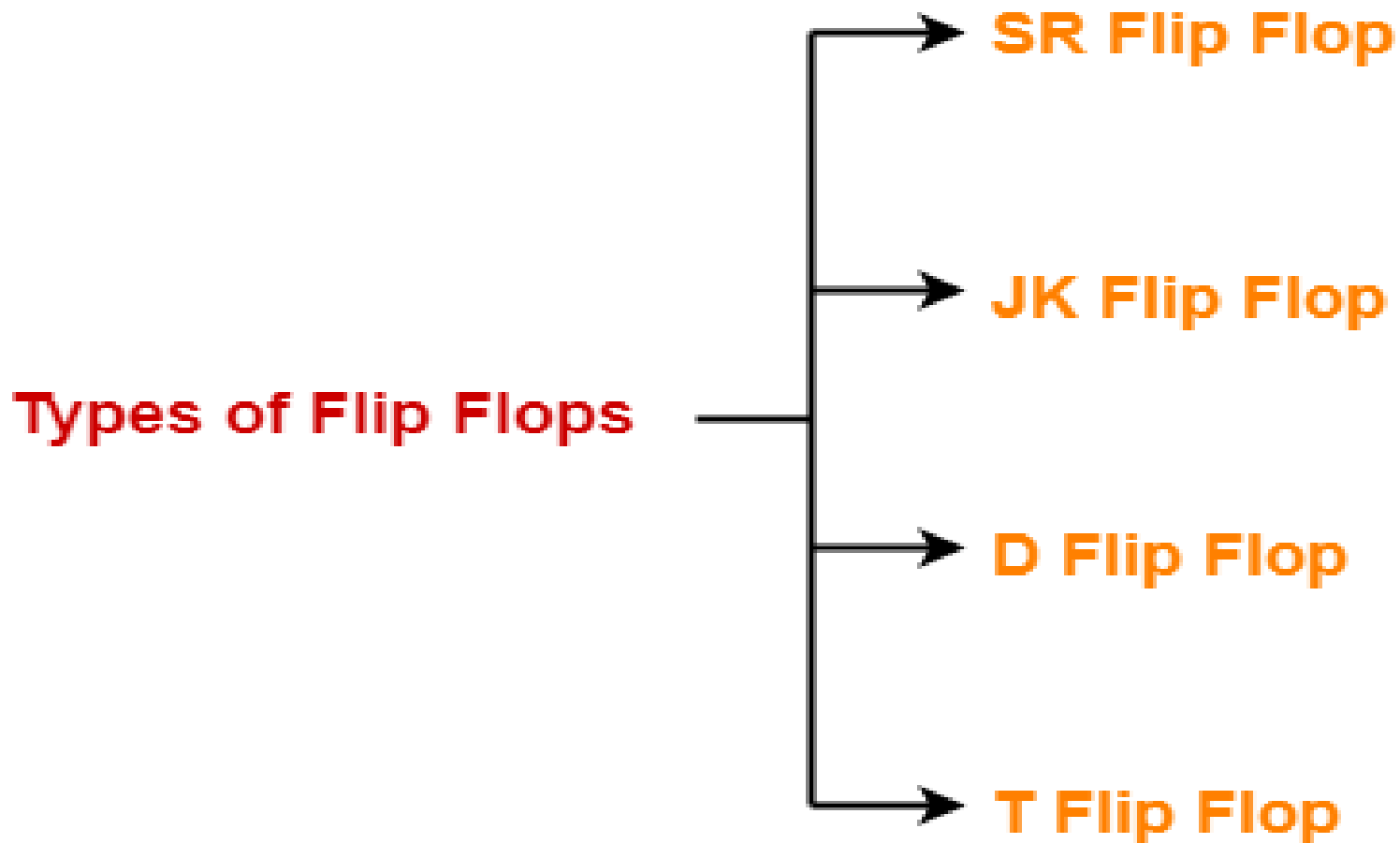
LATCH VS FLIP FLOP

Latch	Flip-flop
It refers to non-clocked flip-flops, because these flip-flops "latch on" to a "1" or a "0" immediately upon receiving the input pulse called SET or RESET.	It samples the inputs only at a clock event.
Latches are level sensitive. 	Flip-flops are edge sensitive. 
Latches are not dependent on the clock signal for their operation, which means a latch is a sequential device that checks all its inputs continuously and changes its outputs accordingly at any time independent of clock signal.	Flip-flop depends on clock signal and continuously checks its inputs and corresponding changes its output only at times determined by clock signal.
It works based on enable function input.	It works on the basis of clock pulse.

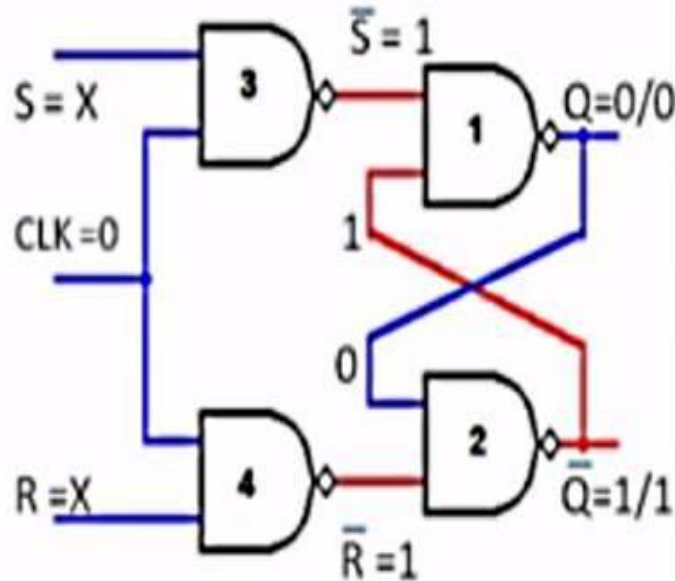
TRIGGERING SIGNALS FOR LATCH AND FF



CLASSIFICATION OF FLIP FLOPS



TRUTH TABLE OF CLOCKED SR FLIP FLOP



INPUT					OUTPUT		STATE OF THE OUTPUT
CLK	S	R	$\bar{S}$	$\bar{R}$	Q_{n+1}	$\bar{Q}_{n+1}$	
0	X	X	1	1	Q_n	$\bar{Q}_{n+1}$	HOLD NO CHANGE PREVIOUS STATE
1	0	0	1	1			
1	0	1	1	0	0	1	RESET
1	1	0	0	1	1	0	SET
1	1	1	0	0	1	1	INVALID FORBIDDEN

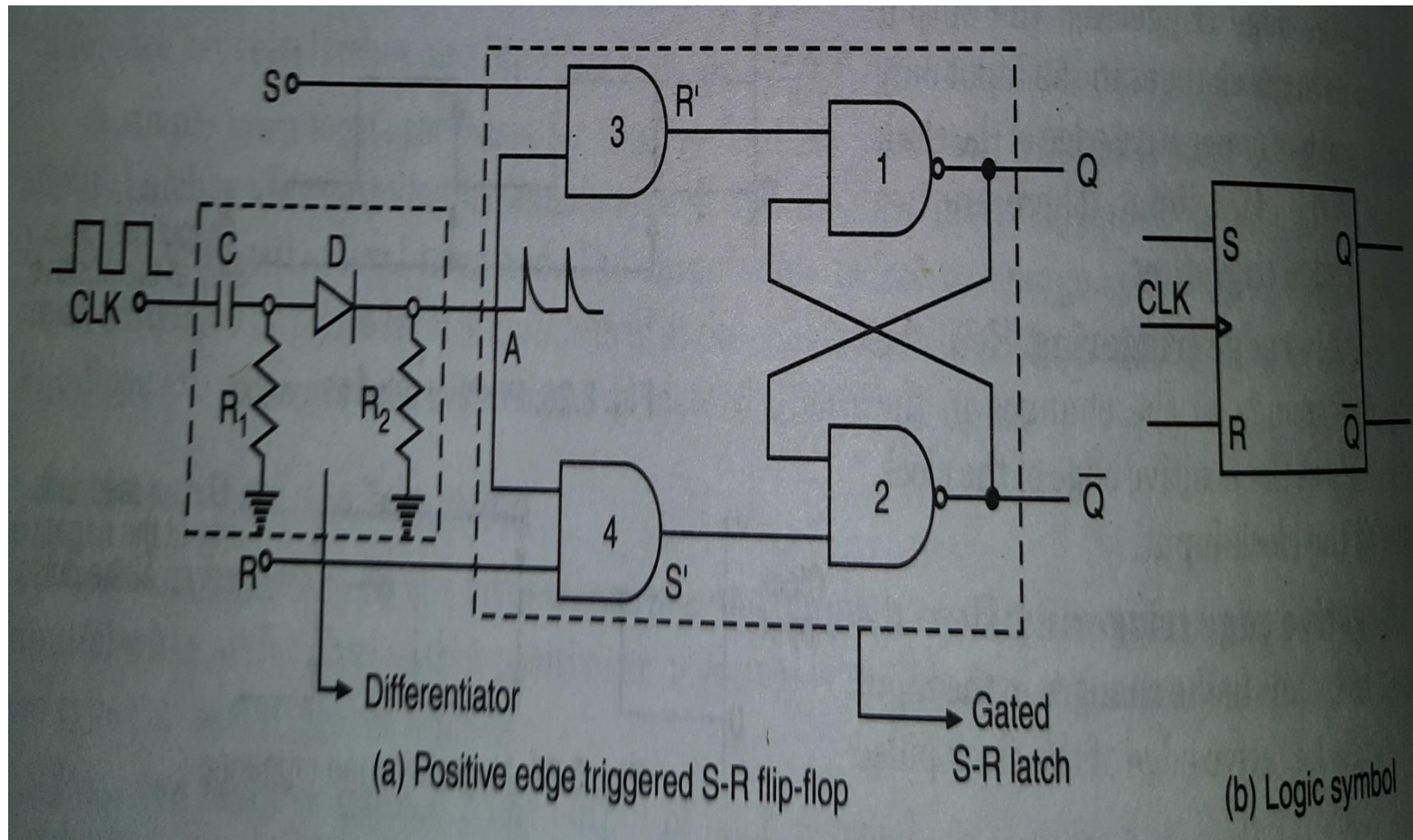
WORKING:

CASE (i): WHEN $CLK = 0, S=R=X \Rightarrow Q_{n+1} = Q_n, \bar{Q}_{n+1} = \bar{Q}_n$

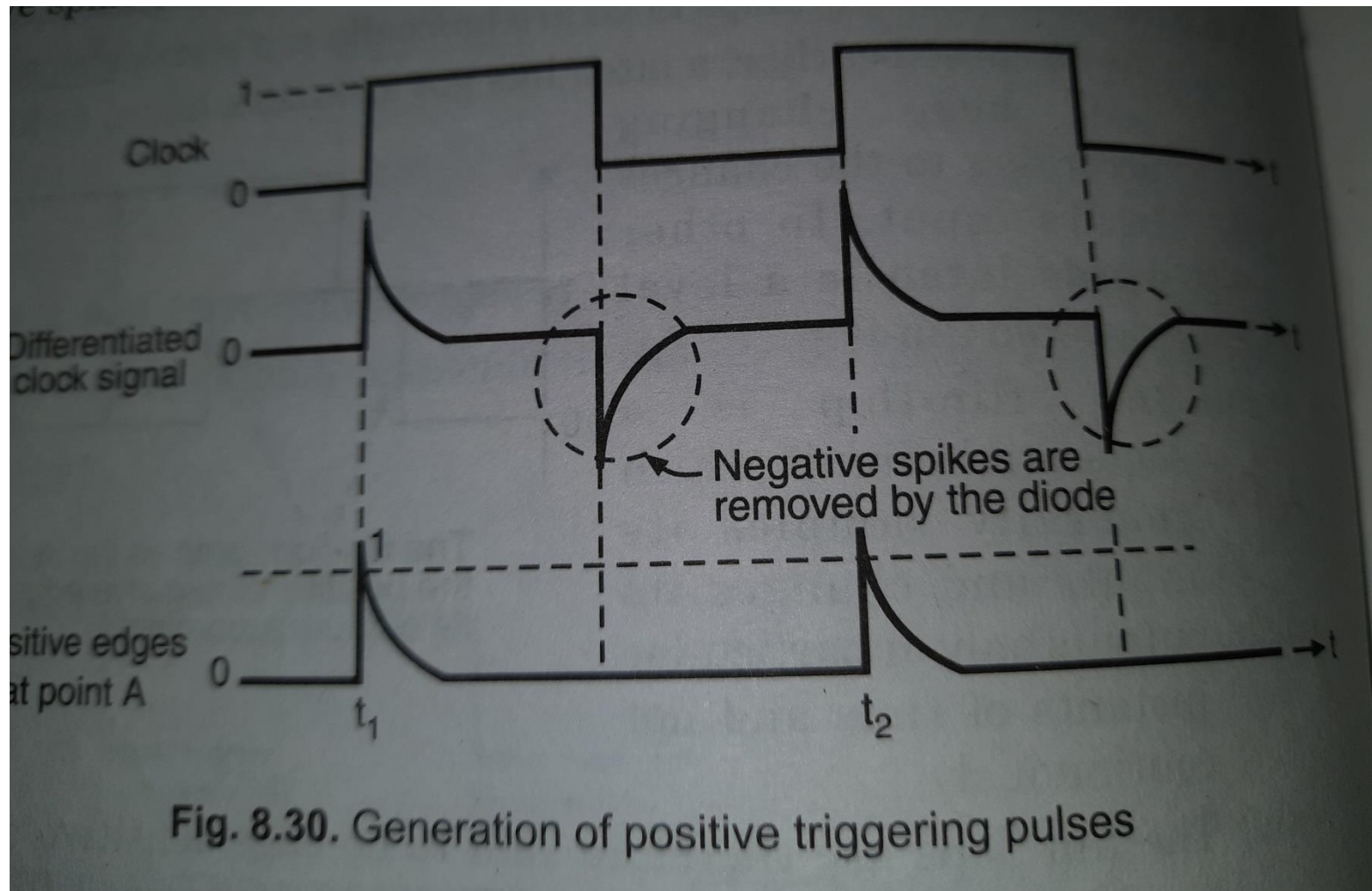
When $clk = 0$, NAND gate 3 & 4 output will be 1 i.e. $\bar{S} = \bar{R} = 1$. The output of gate 1 & 2 now depends on the past output. If $Q = 0, \bar{Q} = 1$ initially then one of the input to gate 2 will be at low state which forces the output of gate 2 to be in the previous state only i.e. $\bar{Q} = 1$ and now both the inputs to gate 1 are at high state hence the output of gate one is forced to be in the previous state only i.e. $Q = 0$.

hence this state is called as HOLD state / NO CHANGE state / PREVIOUS state

POSITIVE EDGE TRIGGERED RS FF



POSITIVE EDGE CLOCK PULSES



TRUTH TABLE

Inputs			Outputs		Remark
CLK	S	R	Q_{n+1}	$\bar{Q}_{n+1}$	
0	x	x	Q_n	$\bar{Q}_n$	No change (NC)
1	x	x	Q_n	$\bar{Q}_n$	No change (NC)
↓	x	x	Q_n	$\bar{Q}_n$	No change (NC)
↑	0	0	Q_n	$\bar{Q}_n$	No change (NC)
↑	0	1	0	1	Reset
↑	1	0	1	0	Set
↑	1	1	Race	Race	Avoid

FF is disabled {

FF responds only to the positive edges {

= Negative edge of clock, ↑ = Positive edge of clock

NEGATIVE EDGE TRIGGERED RS FF

304

Table 8.3. Truth table of a negative edge triggered S-R FF

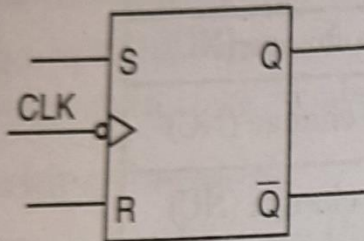


Fig. 8.31. Circuit symbol of negative edge triggered SR FF

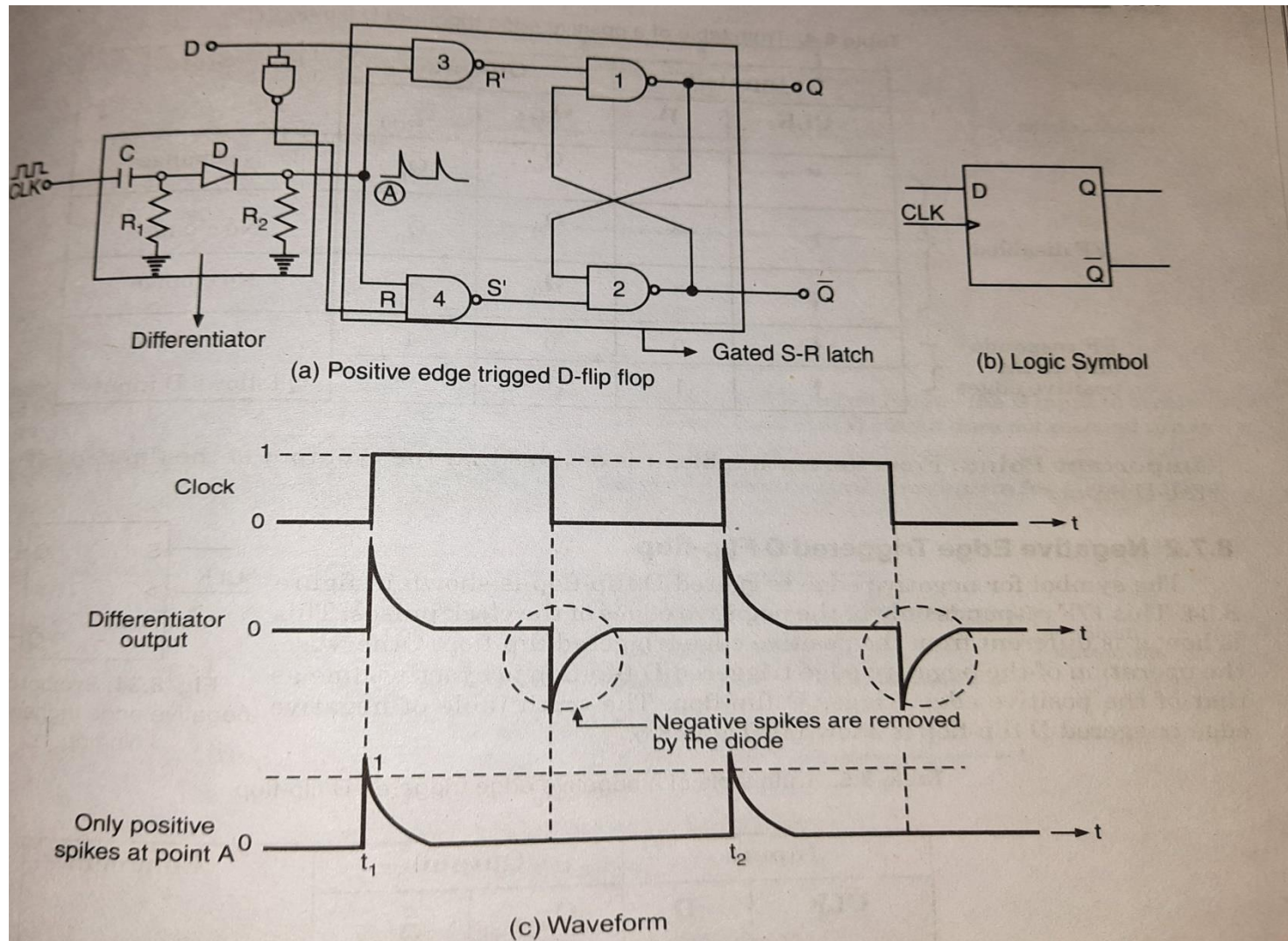
Here $\uparrow$ = Positive edge of clock
 $\downarrow$ = Negative edge of clock

Inputs			Outputs		Remark
CLK	S	R	Q_{n+1}	$\bar{Q}_{n+1}$	
0	x	x	Q_n	$\bar{Q}_n$	No change (NC)
1	x	x	Q_n	$\bar{Q}_n$	No change (NC)
$\uparrow$	x	x	Q_n	$\bar{Q}_n$	No change (NC)
$\downarrow$	0	0	Q_n	$\bar{Q}_n$	No change (NC)
$\downarrow$	0	1	0	1	Reset
$\downarrow$	1	0	1	0	Set
$\downarrow$	1	1	Race	Race	Avoid

FF is disabled

FF responds only to negative edges

POSITIVE EDGE TRIGGERED D FF



TRUTH TABLE OF POSITIVE EDGE TRIGGERED D FF

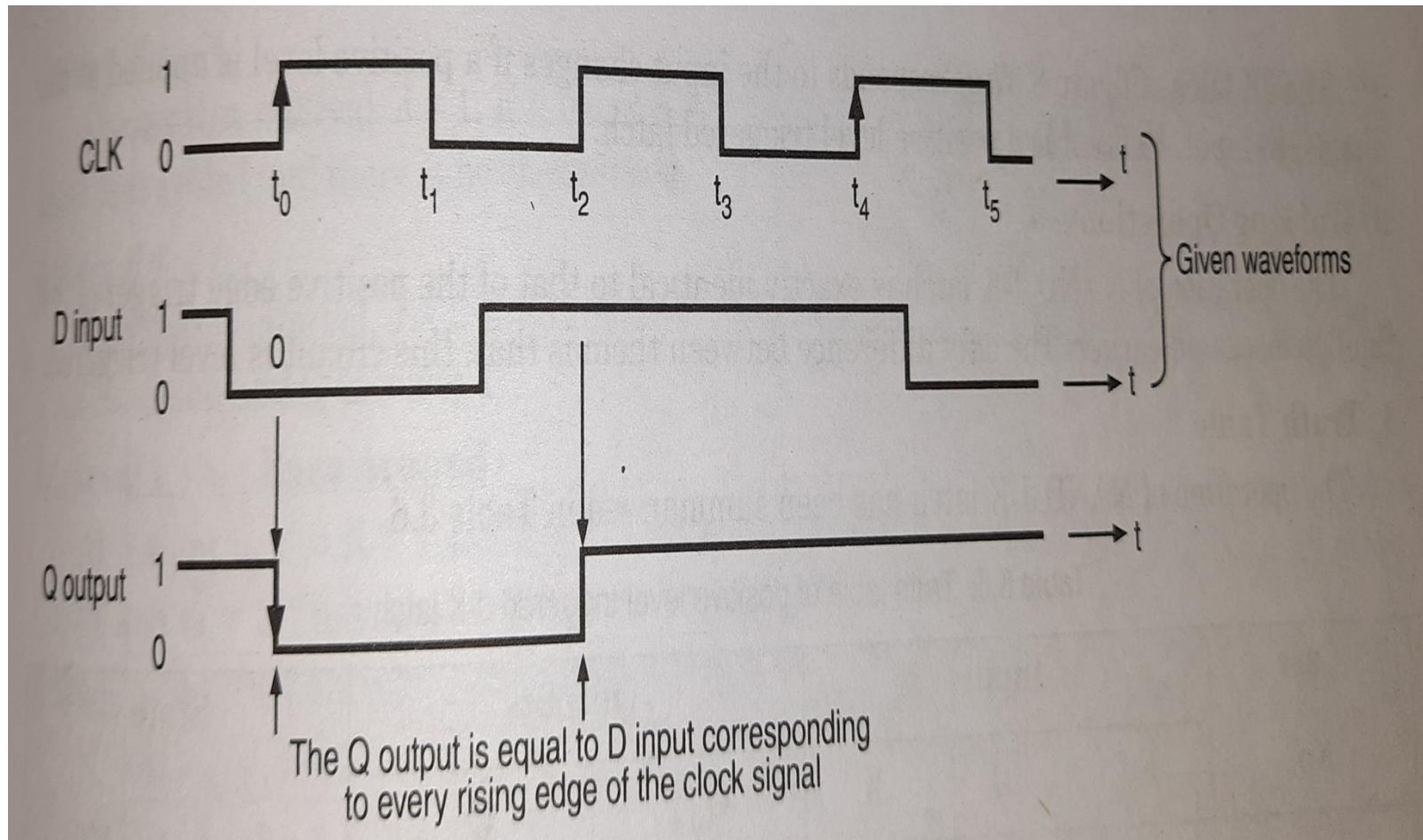
Table 8.4. Truth table of a positive edge triggered D flip-flop

Inputs		Outputs		State
CLK	D	Q_{n+1}	$\bar{Q}_{n+1}$	
0	x	Q_n	$\bar{Q}_n$	No change
1	x	Q_n	$\bar{Q}_n$	No change
↓	x	Q_n	$\bar{Q}_n$	No change
↑	0	0	1	Q follows D input
↑	1	1	0	

FF disabled {

FF responds only to the positive edges {

TIMING DIAGRAM OF POSITIVE EDGE TRIGGERED D FF



NEGATIVE EDGE TRIGGERED D FF

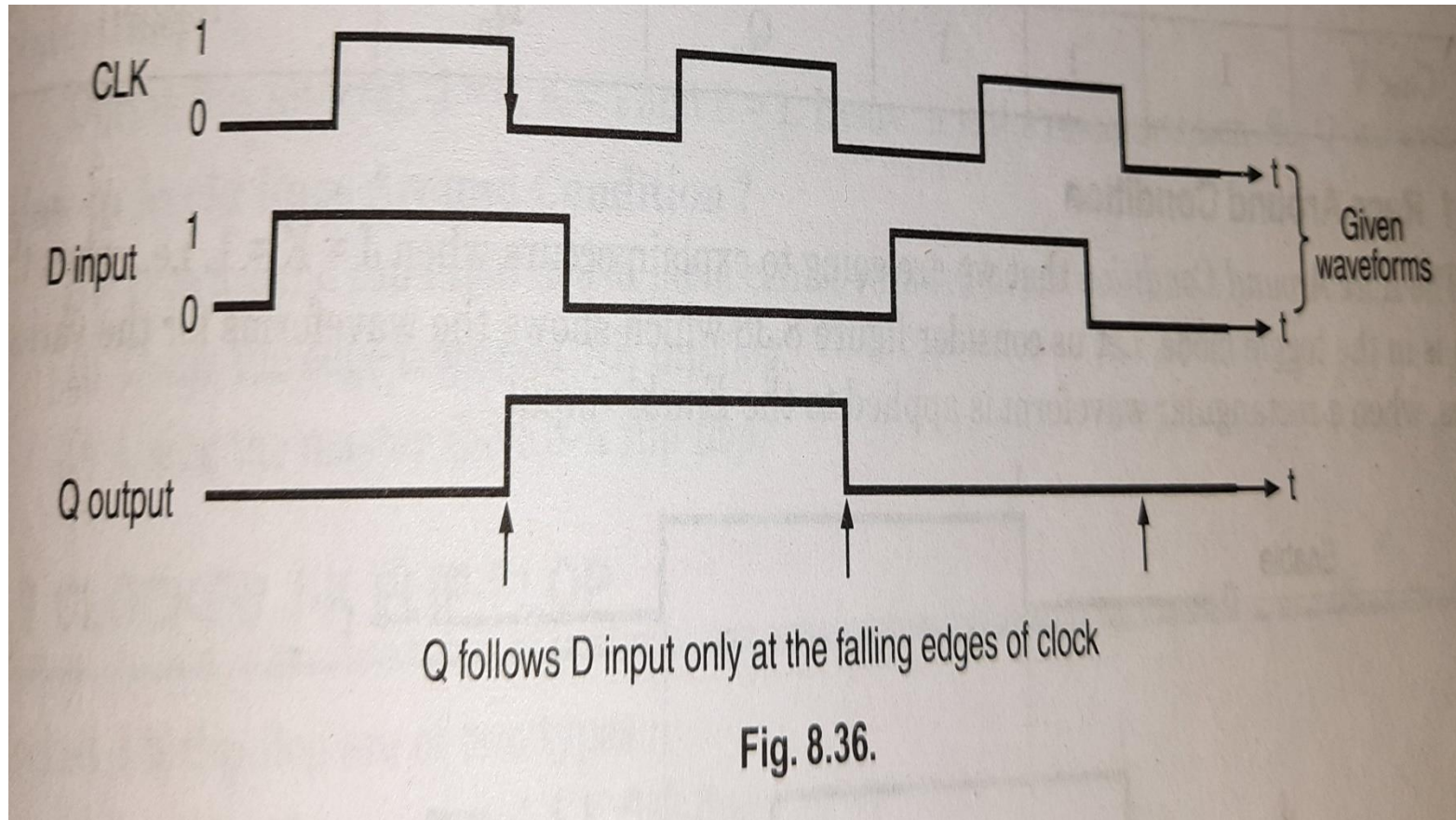
Table 8.5. Truth table of a negative edge triggered D flip-flop

Inputs		Outputs		Comment
CLK	D	Q_{n+1}	$\bar{Q}_{n+1}$	
0	x	Q_n	Q_n	No change
1	x	Q_n	$\bar{Q}_n$	No change
↑	x	Q_n	$\bar{Q}_n$	No change
↓	0	0	1	Q output follows D input
↑	1	1	0	

FF disabled {

FF responds only to the positive edges {

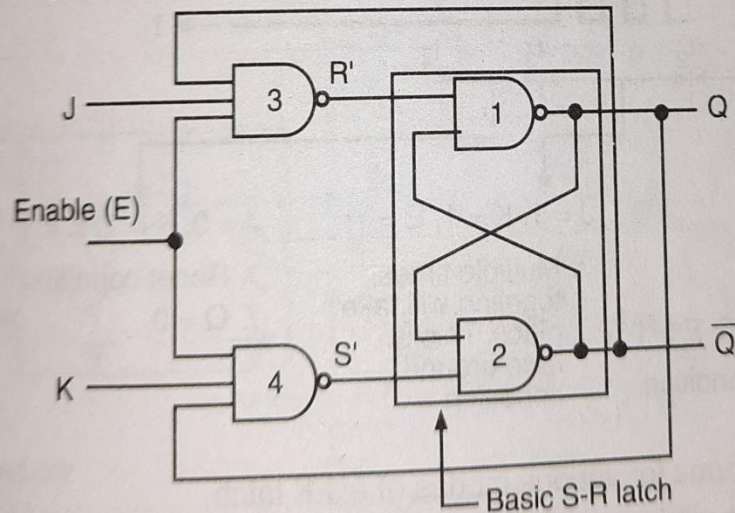
TIMING DIAGRAM OF NEGATIVE EDGE TRIGGERED D FF



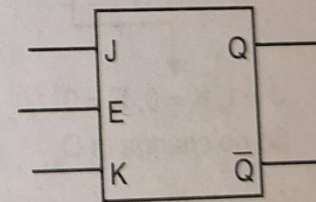
J-K LATCH

1. Circuit Diagram and Circuit Symbol

The JK latch using NAND gate is shown in figure 8.37. It consists of the basic SR latch and an enable input. Note that the output Q and $\bar{Q}$ are returned back and connected to the inputs of NAND gates 4 and 3 respectively.



(a) A J-K latch



(b) Symbol of the J-K latch

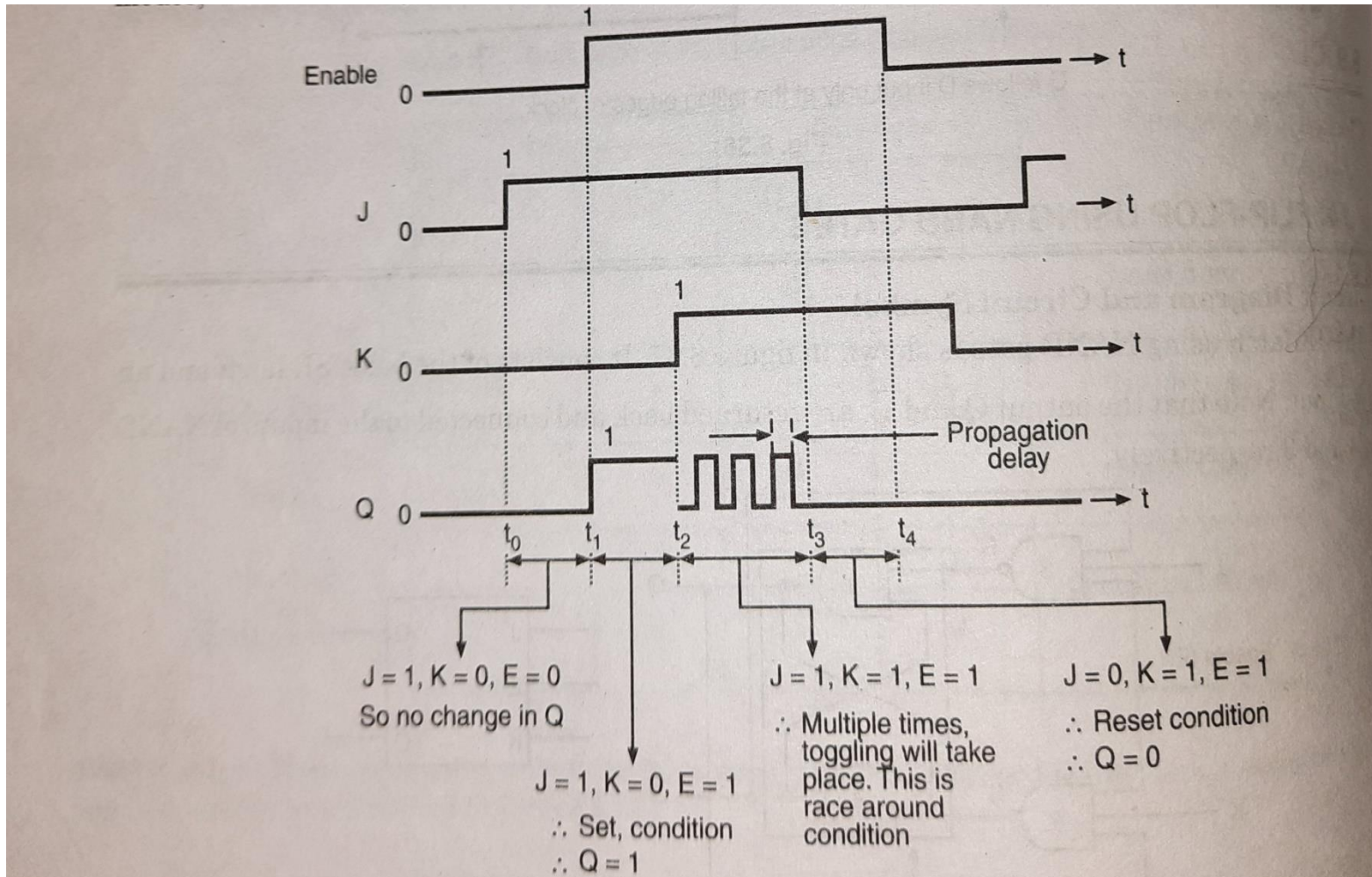
Fig. 8.37.

TRUTH TABLE OF POSITIVE LEVEL TRIGGERED J-K LATCH

Table 8.6. Truth table of positive level triggered J-K latch

Case No.	Inputs			Outputs		State
	E	J	K	Q_{n+1}	$\bar{Q}_{n+1}$	
Case I	0	x	x	Q_n	$\bar{Q}_n$	No change
Case II	1	0	0	Q_n	$\bar{Q}_n$	No change
Case III	1	0	1	0	1	Reset
Case IV	1	1	0	1	0	Set
Case V	1	1	1	$\bar{Q}_n$	Q_n	Toggle

RACE AROUND CONDITION



J-K FF- POSITIVE EDGE TRIGGERED

8.9 CLOCKED J-K FLIP-FLOP

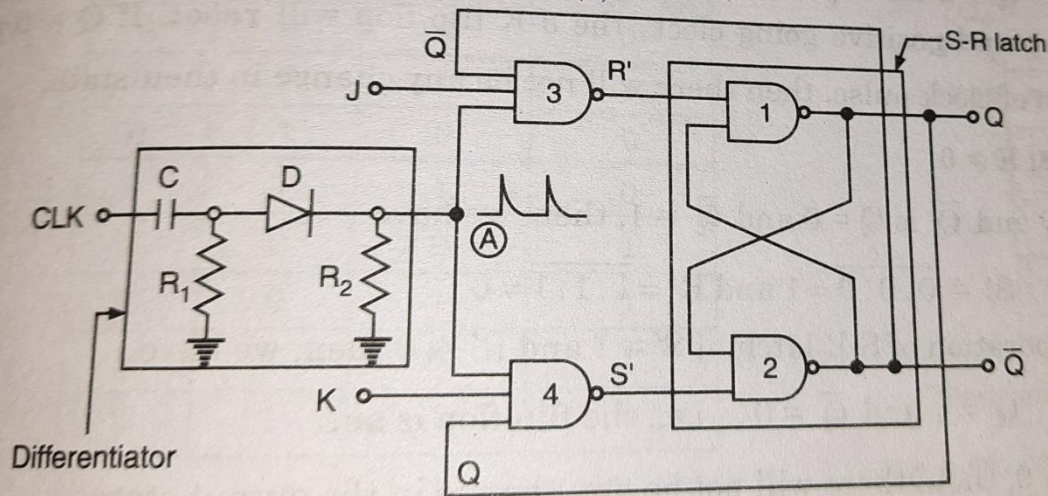
Clocked J-K flip-flop are of two types :

- (i) Positive edge triggered J-K flip-flop
- (ii) Negative edge triggered J-K flip-flop

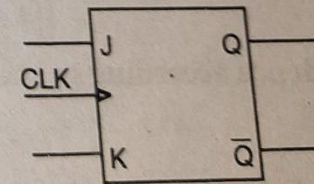
8.9.1. Positive Edge Triggered J-K Flip-flop

1. Circuit Diagram and Circuit Symbol

The circuit diagram of the positive edge triggered J-K flip-flop is shown in figure 8.39 (a) and its circuit symbol is shown in figure 8.39 (b).



(a) Positive edge triggered J-K flip-flop



(b) Logic Symbol

Fig. 8.39.

WORKING OF JK FF

2. Working Operation

The clock signal is a train of rectangular or square waves. It is passed through a differentiator ($C - R_1$) and a rectifier (D and R_2), to obtain the positive spikes at point A as already discussed. NAND gates 1 and 2 form an S-R latch. The other two NAND gates have three inputs each. The outputs Q and $\bar{Q}$ are fed back to the inputs of gates 4 and 3. According to figure 8.39 (a), we can write the mathematical expressions for S' and R' as under :

$$S' = \overline{K \cdot Q \cdot CLK} \text{ and } R' = \overline{J \cdot Q \cdot CLK}$$

3. Working Operation

Let us now understand the operation step-by-step.

(i) Case I : CLK = 0 or 1, i.e., level

Since CLK = 0 or 1, i.e., level, this flip-flop is disabled, because the output of differentiator is zero for any level input. Therefore Q and $\bar{Q}$ output do not change their state.

(ii) Case II : If clock edge is falling edge

For the falling edge of the clock, the rectifier ($D - R_2$) will block the negative spikes. Hence, voltage at point A is logic 0. So, one input to both the NAND gate 3 and 4 is 0. Hence, both S' and R' will be forced to 1. For an S-R latch using NAND gates, the outputs will remain unchanged if $S = R = 1$.

Therefore, corresponding to the falling edge of clock, the outputs Q and $\bar{Q}$ will remain unchanged.

(iii) Case III : CLK = $\uparrow$, J = 0, K = 0

Since $J = 0$, $R' = 1$ and as $K = 0$, $S' = 1$. Since, $S' = R' = 1$, the outputs Q and $\bar{Q}$ will not change their state even though a positive edge of the clock pulse is being applied.

(iv) Case IV : CLK = $\uparrow$, J = 0, K = 1

Recalling the expressions for S' and R' , we have

$$S' = \overline{K \cdot Q \cdot CLK} \text{ and } R' = \overline{J \cdot Q \cdot CLK}$$

If the previous state of Q and $\bar{Q}$ is $Q = 1$ and $\bar{Q} = 0$, then,

$$S' = \overline{1 \cdot 1 \cdot 1} = 0 \text{ and } R' = \overline{0 \cdot 0 \cdot 1} = 1$$

Hence according to the operation of S-R latch, if $S' = 0$, $R' = 1$, then $Q = 0$, and $\bar{Q} = 1$.

Therefore,

$$Q = 0 \text{ and } \bar{Q} = 1$$

Thus, with $J = 0$, $K = 1$ and positive going clock, the J-K flip-flop will **reset**. If $Q = 0$ and $\bar{Q} = 1$ before the application of clock pulse, then there will not be any change in their state.

(v) Case V : CLK = $\uparrow$, J = 1, K = 0

If the previous state of Q and $\bar{Q}$ is $Q = 0$ and $\bar{Q} = 1$, then, we have

$$S' = \overline{0 \cdot 0 \cdot 0} = 1 \text{ and } R' = \overline{1 \cdot 1 \cdot 1} = 0$$

Hence according to the operation of S-R latch, if $S' = 1$ and $R' = 0$ then, we have

$$Q = 1 \text{ and } \bar{Q} = 0 \dots \text{i.e., the flip-flop is set.}$$

If Q and $\bar{Q}$ are already 1, 0, then, there will not be any change in the output state.

An idealist is one who helps other people to be properous.

— Henry Ford

(vi) Case VI : CLK = $\uparrow$, J = 1, K = 1 (Toggle Mode)

We shall discuss this case for two different previous cases.

- Previously let $Q = 1$ and $\bar{Q} = 0$

↓

- $S' = \overline{K \cdot Q \cdot CLK}$ and $R' = \overline{J \cdot Q \cdot CLK} = 1$

↓

$$\therefore S' = 1 \cdot 1 \cdot 1 = 0 \text{ and } R' = 1 \cdot 0 \cdot 1 = 1$$

↓

- Hence, the SR latch will reset

↓

- So, $Q_{n+1} = 0$ and $\bar{Q}_{n+1} = 1$

- Previously let $Q = 0$ and $\bar{Q} = 1$

↓

- $S' = 1 \cdot 0 \cdot 1$ and $R' = 1 \cdot 1 \cdot 1$

↓

- So, $S' = 1$ and $R' = 0$

↓

- Hence, the SR latch will set

↓

- So, $Q_{n+1} = 1$ and $\bar{Q}_{n+1} = 0$

From the operation discussed above, we conclude that when J = K = 1 and clock hits, then Q and $\bar{Q}$ outputs are inverted, i.e., $Q_{n+1} = \bar{Q}_n$ and $\bar{Q}_{n+1} = Q_n$. This is called as the TOGGING mode. This is a very important mode of operation.

Table 8.7. Truth table for positive edge triggered J-K FF

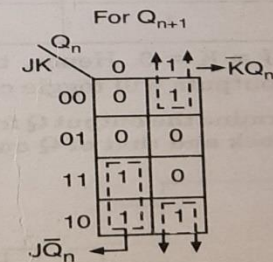
Case No.	Inputs			Outputs		State
	CLK	J	K	Q_{n+1}	$\bar{Q}_{n+1}$	
Case I	0 or 1	x	x	No change	No change	Flip-flop is disaled
Case II	↓	x	x	No change	No change	
Case III	↑	0	0	No change	No change	
Case IV	↑	0	1	0	1	Reset
Case V	↑	1	0	1	0	Set
Case VI	↑	1	1	$\bar{Q}_n$	Q_n	Toggle

8.9.2 Characteristic Equation of J-K Flip-flop

Let us consider the truth table of J-K flip-flop (Table 8.8) and write the K-map for Q_{n+1} shown in figure 8.40.

Table 8.8. Truth table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



$$\text{Therefore, } Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Fig. 8.40. K-map for $Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$.

NO RACE AROUND CONDITION IN EDGE TRIGGERED J-K FF

- For racing around to take place, it is essential to have the enable input high along with $J=K=1$
- As enable input remains high for a long time in J-K latch, the problem of multiple toggling arises.
- But in edge triggered J-K FF, positive clock pulse is present only for a short time.
- Hence, by the time the changed outputs return back to the inputs of NAND gates 3 and 4, the clock pulse has died down to zero
- Multiple toggling or race around condition is avoided



TOGGLE FF (T – FF)

- T flip-flops operate by toggling the previous output. For example when $T = 0$ it will continue to output Q but when $T = 1$ it will output $\overline{Q}$.

T Flip-flop

Symbol

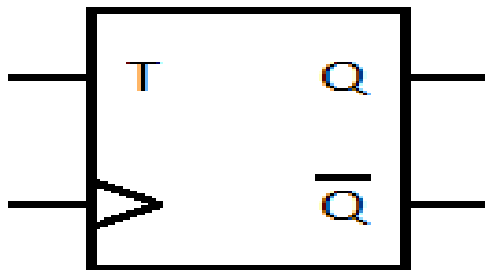


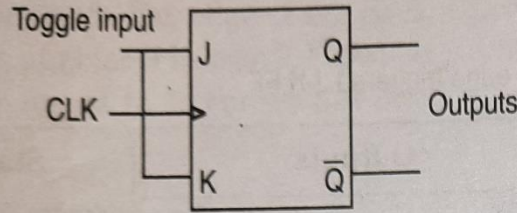
Table of truth:

T	Q	$\overline{Q}$
0	Q	$\overline{Q}$
1	$\overline{Q}$	Q
0	$\overline{Q}$	Q
1	Q	$\overline{Q}$

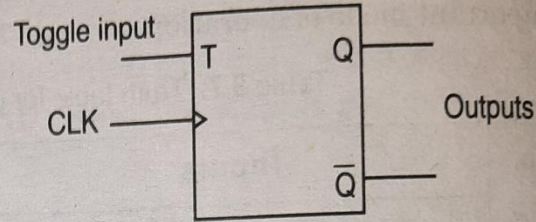


1. Circuit Diagram and Logic Symbol

Toggle flip-flop is basically a J-K flip-flop with J and K terminals permanently connected together. It has only one input denoted by T, as shown in figure 8.41(a). The symbol for positive edge triggered T flip-flop is shown in figure 8.41(b) and Table 8.9 shows its truth table.



(a) JK FF is converted into T flip-flop



(b) Logic symbol of positive edge triggered T flip-flop

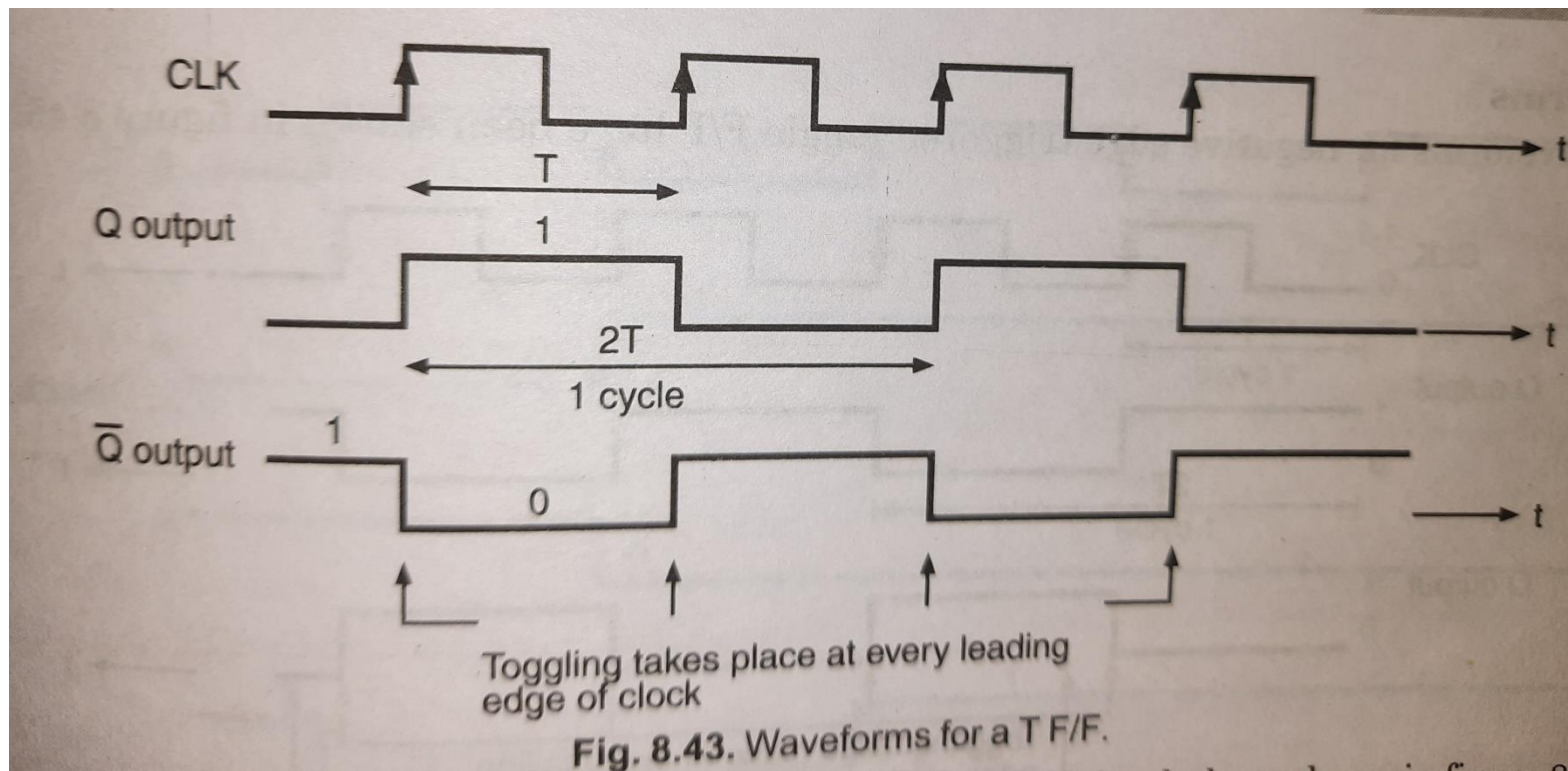
Fig. 8.41.

Table 8.9. Truth table of a toggle FF (positive edge triggered)

Inputs		Outputs		State
CLK	T	Q_{n+1}	$\bar{Q}_{n+1}$	
↑	0	Q_n	$\bar{Q}_n$	No change
↑	×	Q_n	$\bar{Q}_n$	No change
1	×	Q_n	$\bar{Q}_n$	No change
0	×	Q_n	$\bar{Q}_n$	No change
↑	1	$\bar{Q}_n$	Q_n	Toggle

When $T = 0$, $J = K = 0$. Hence, the outputs Q and $\bar{Q}$ will not change. But, if $T = 1$, then $J = K = 1$, and the outputs will toggle corresponding to every leading edge of clock signal.

WAVEFORMS OF T FF: POSITIVE EDGE TRIGGERED



Relation between Input and Output Frequency

Referring to figure 8.43, we have

1 cycle period of CLK signal = T

Therefore, clock frequency $f_{CLK} = \frac{1}{T}$

1 cycle period of Q output = $2T$

Hence, output frequency $f_0 = \frac{1}{2T}$

But, $\left(\frac{1}{T}\right) = f_{CLK}$

Therefore, output frequency $f_0 = \frac{f_{CLK}}{2}$



8.10.1 Negative Edge Triggered T Flip-flop

1. Logic symbol and Truth Table

Figure 8.44 shows the logic symbol of a negative edge triggered toggle flip-flop and Table 8.10 gives its truth table.

Table 8.10. Truth Table

Inputs		Outputs		State
T	CLK	Q_{n+1}	$\overline{Q}_{n+1}$	
0	$\times$	Q_n	$\overline{Q}_n$	No change
1	$\uparrow$	Q_n	$\overline{Q}_n$	No change
1	0	Q_n	$\overline{Q}_n$	No change
1	1	Q_n	$\overline{Q}_n$	No change
1	$\downarrow$	$\overline{Q}_n$	Q_n	Toggle

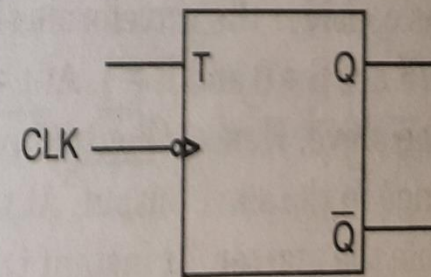
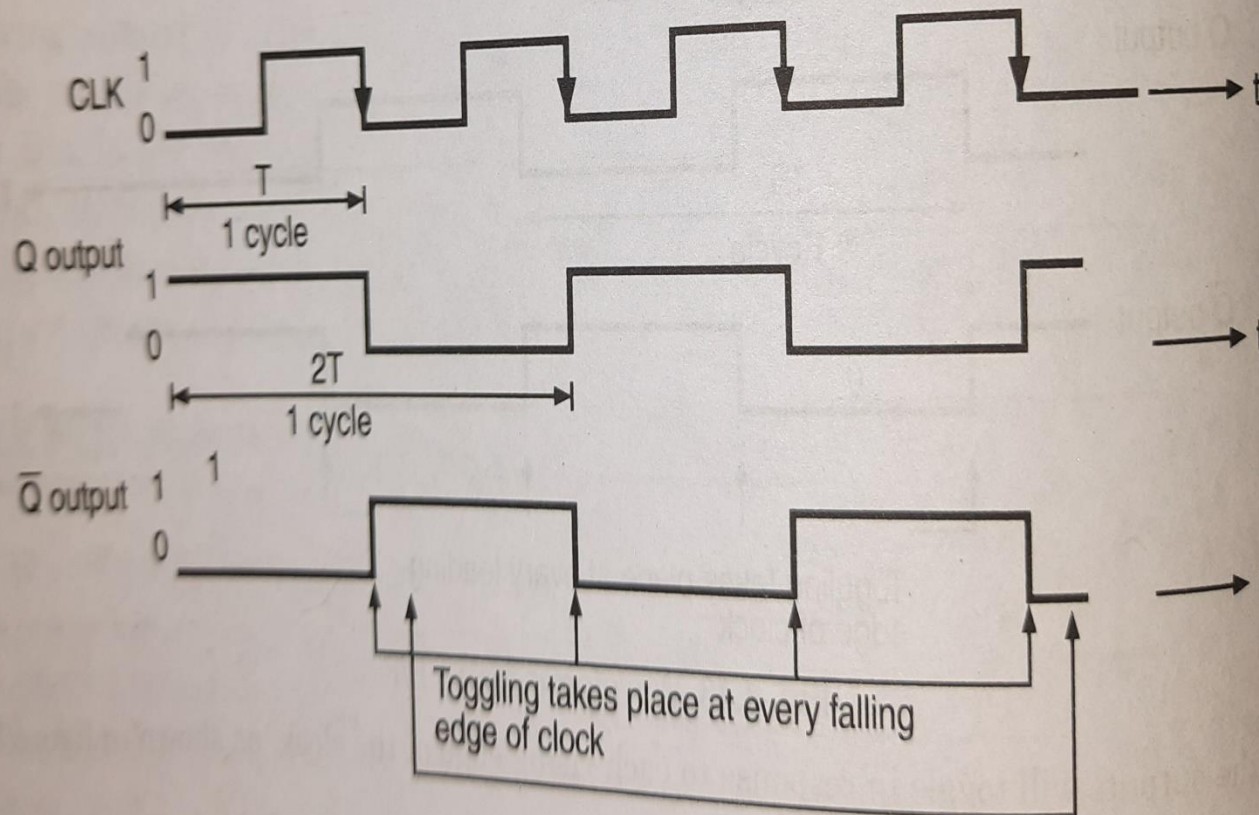


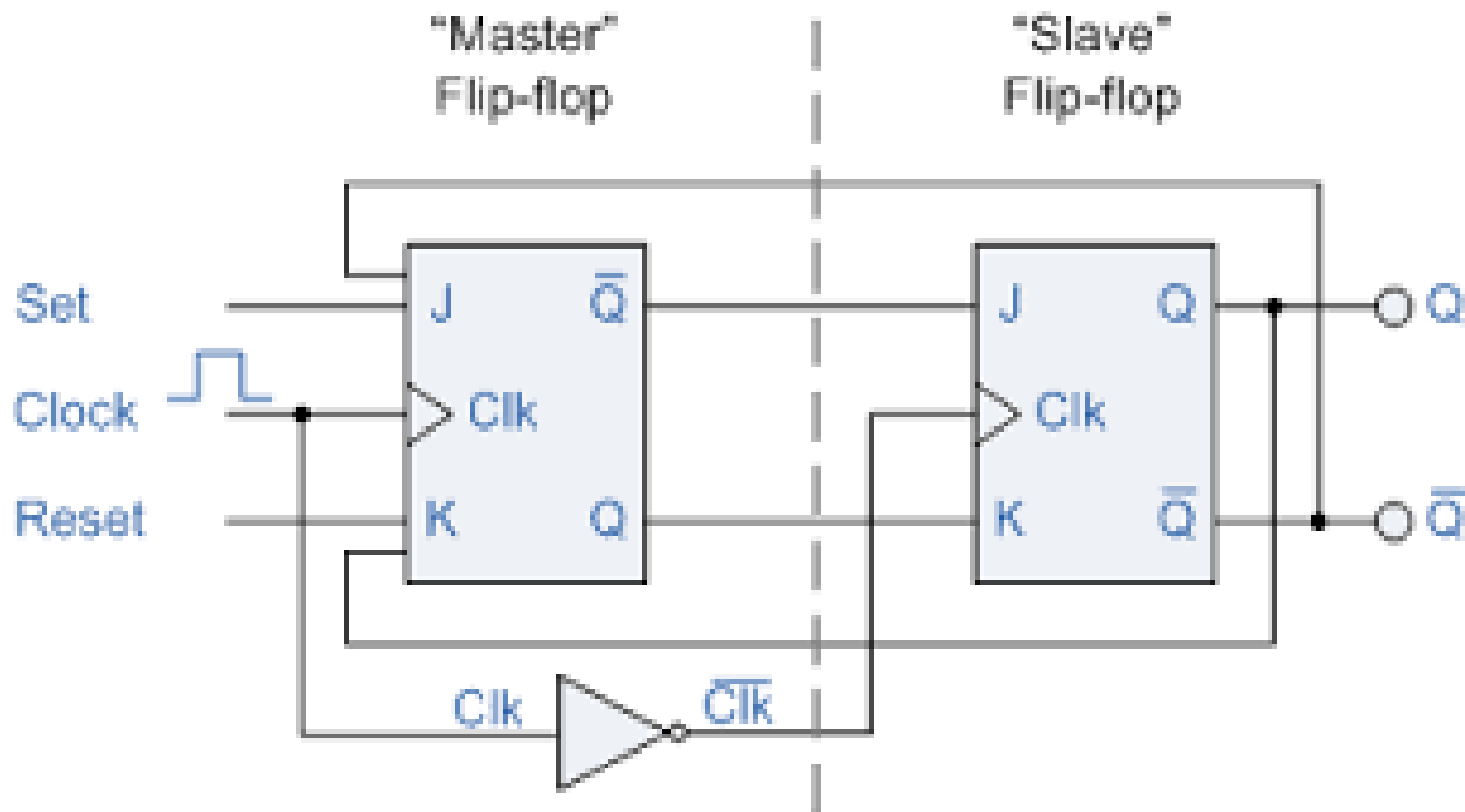
Fig. 8.44. Logic symbol of a negative edge triggered toggle flip-flop.

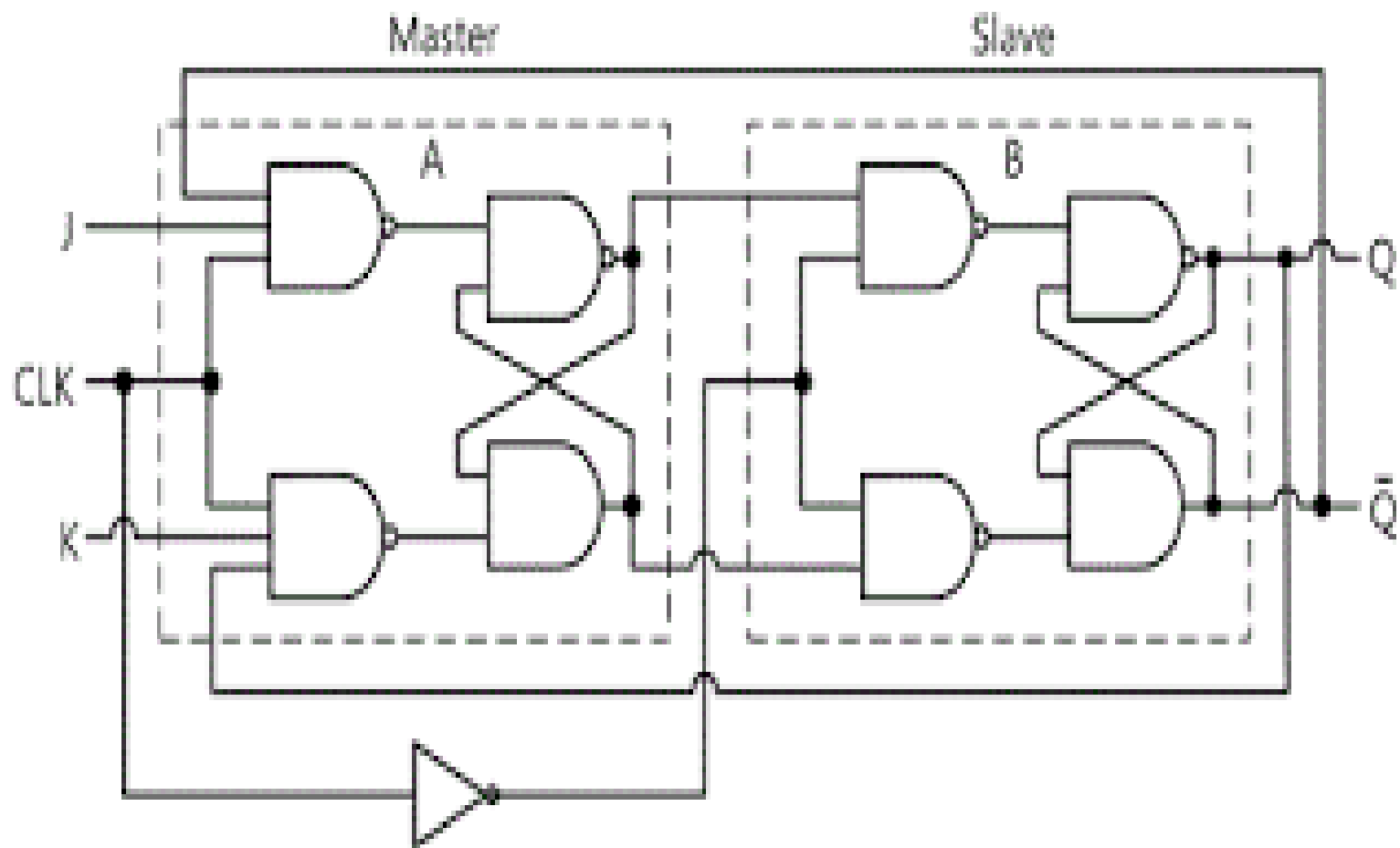
2. Waveforms

The waveforms for negative edge triggered toggle F/F have been shown in figure 8.45.



MASTER SLAVE JK FF





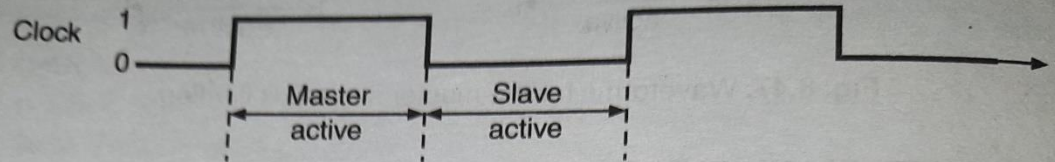


Fig. 8.49.

Case I : Clock = $\times$, $J = K = 0$

- (i) For clock = 1, the master is active, slave inactive. As $J = K = 0$, therefore, outputs of master, i.e., Q_1 and $\bar{Q}_1$ will not change. Hence, the S and R inputs to the slave will remain unchanged.
- (ii) As soon as clock = 0, the slave becomes active and master is inactive. But, since, the S and R inputs have not changed, the slave outputs will also remain unchanged. Therefore, outputs will not change if $J = K = 0$.

Case II : Clock = $\square$, $J = K = 0$

This condition has been already discussed in case I.

Case III : Clock = $\square$, $J = 0$ and $K = 1$

Clock = 1 : Master active, slave inactive.

Therefore, outputs of the master become $Q_1 = 0$ and $\bar{Q}_1 = 1$. This means that $S = 0$ and $R = 1$.

Clock = 0 : Slave active, master inactive.

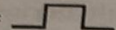
Therefore, outputs of the slave become $Q = 0$ and $\bar{Q} = 1$.

Again, if clock = 1 : Master active, slave inactive.

Therefore, even with the changed outputs $Q = 0$ and $\bar{Q} = 1$ fed back to master, its outputs will $Q_1 = 0$ and $\bar{Q}_1 = 1$. This means that $S = 0$ and $R = 1$.

Hence, with clock = 0 and slave becoming active, the outputs of slave will remain $Q = 0$ and $\bar{Q} = 1$.

Thus, we get a stable output from the Master slave.

Case IV : CLK = , J = 1, K = 0

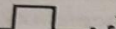
Clock = 1 : Master active, slave inactive.

Therefore, outputs of master become $Q_1 = 1$ and $\bar{Q}_1 = 0$, i.e., $S = 1, R = 0$.

Clock = 0 : Master inactive, slave active.

Therefore, Outputs of slave become $Q = 1$ and $\bar{Q} = 0$.

Again, if clock = 1, then it can be shown that the outputs of the slave are stabilized to $Q = 1$ and $\bar{Q} = 0$.

Case V : CLK = , J = 1, K = 1

Clock = 1 : Master active, slave inactive.

Therefore, Outputs of master will toggle. So, S and R also will be inverted.

Clock = 0 : Master inactive, slave active.

Therefore, Outputs of the slave will toggle.

These changed outputs are returned back to the master inputs.

But since clock = 0, the master is still inactive. So, it does not respond to these changed outputs.

This avoids the multiple toggling which leads to the race around condition. Thus, the master slave flip-flop will avoid the race around condition.

4. Waveforms

The waveforms for the master slave flip-flop have been shown in figure 8.50.

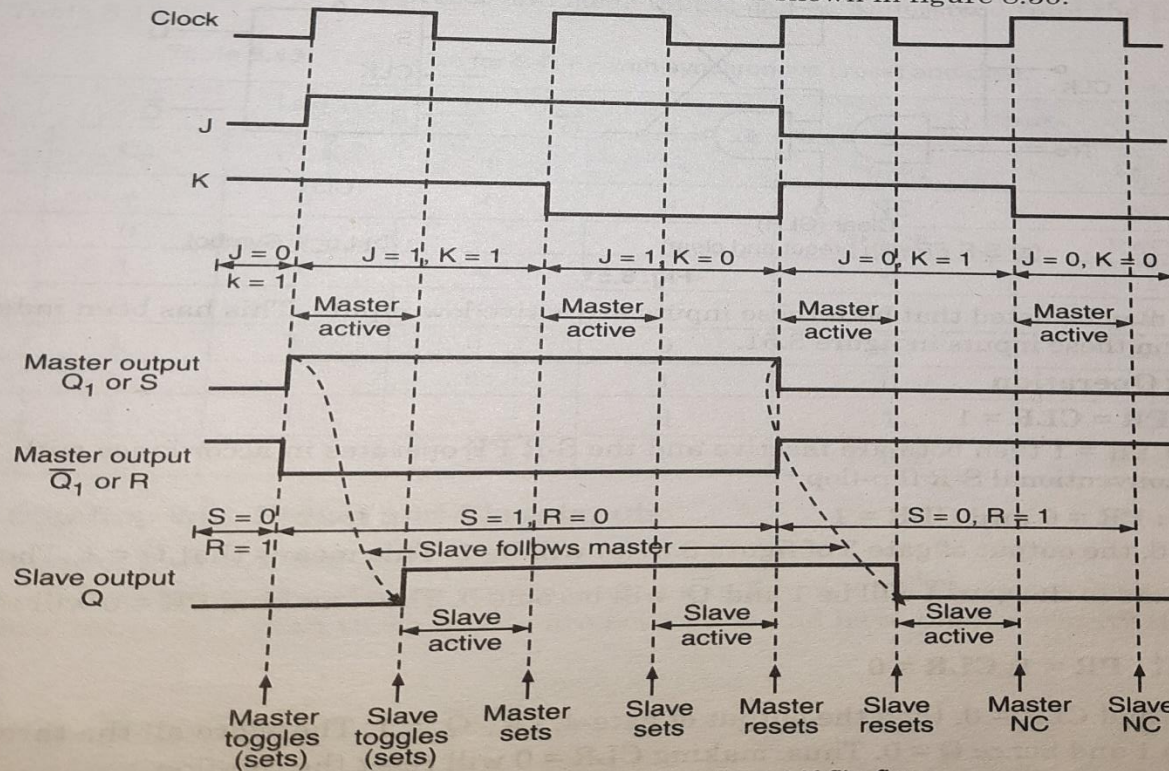


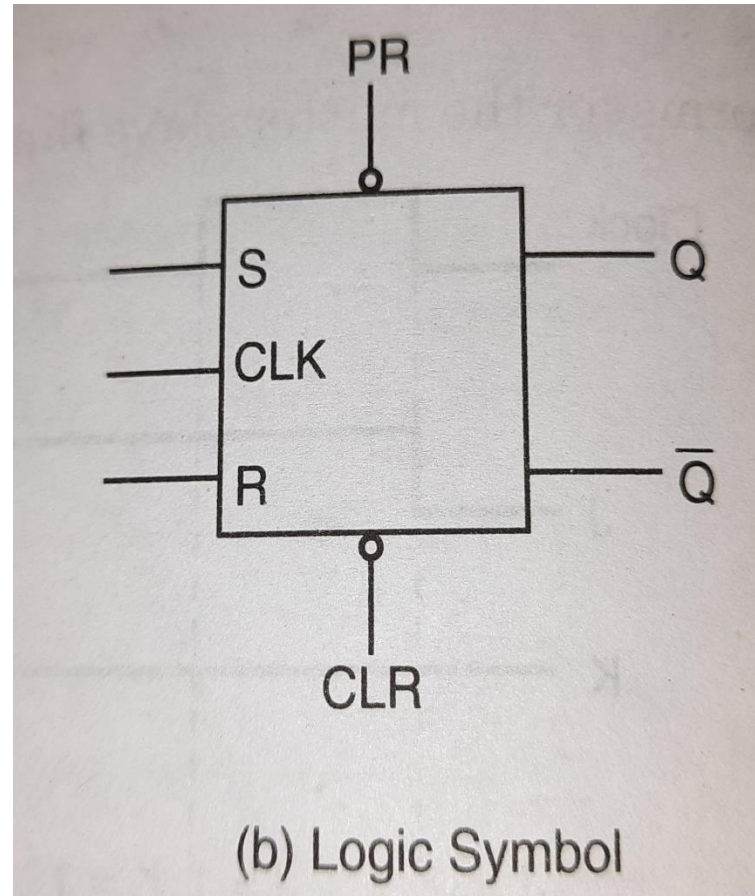
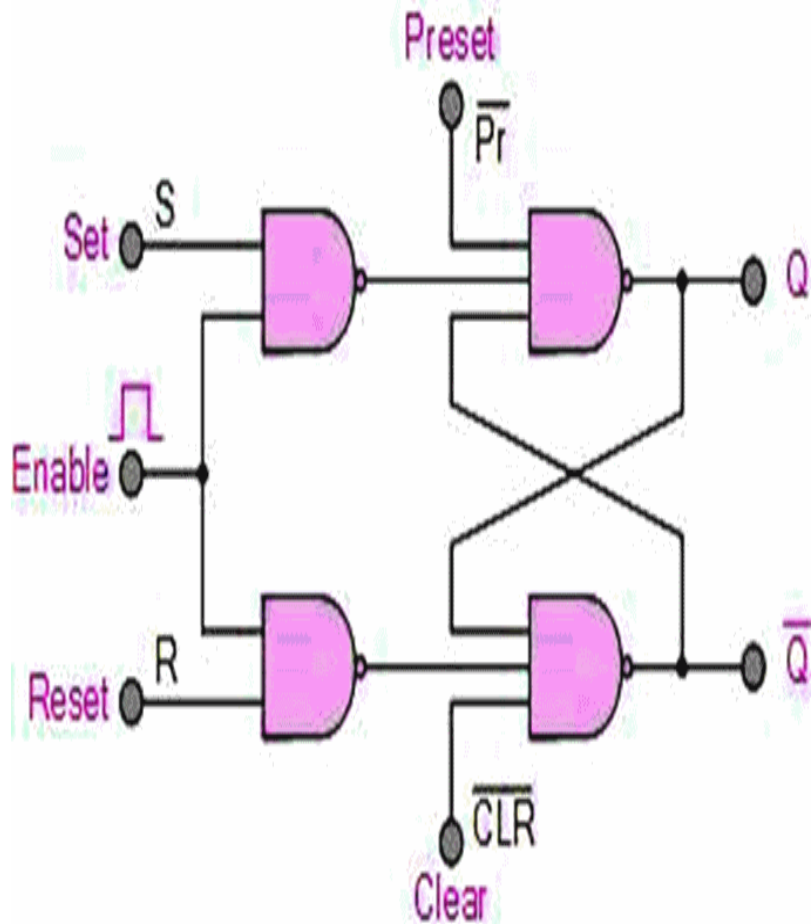
Fig. 8.50 Waveforms of master slave J-K flip-flop.

PRESET AND CLEAR INPUTS

- When power switch is turned on, the state of outputs Q and Q' is uncertain. It can be $Q=0, Q'=1$ or other way around
- In some applications, this uncertainty is not acceptable.
- It is necessary to initially set or reset the FFs
- This can be done by adding two asynchronous inputs as Preset(PR) and Clear(CLR)
- Most clocked FFs have asynchronous inputs that can set or clear the FF independently of the clock input
- Clearing(resetting) a FF means that its output ends up in the $Q = 0$ and $Q' = 1$ state.
- Setting a FF means that its output ends up in $Q = 1$ and $Q' = 0$ state.



SR FF WITH PRESET AND CLEAR INPUTS



S-R FF WITH ASYNCHRONOUS PRESET AND CLEAR INPUTS

Table 8.12. Summary of operation of S-R FF

Inputs			Output	Operation performed
CLK	PR	CLR		
1	1	1	Q_{n+1}	Normal S-R flip-flop
x	0	1	1	FF is set
x	1	0	0	FF is reset

The don't care conditions (x) marked in the CLK column indicate that PR and CLR inputs have higher priority than clock input.

8.13.1 Synchronous Preset and Clear Inputs

1. Basic Concepts

Preset and clear inputs can be of following two types :

- Asynchronous with clock
- Synchronous with clock

Asynchronous preset and clear inputs.



S-R FF WITH SYNCHRONOUS PRESET AND CLEAR INPUTS WITH CLOCK

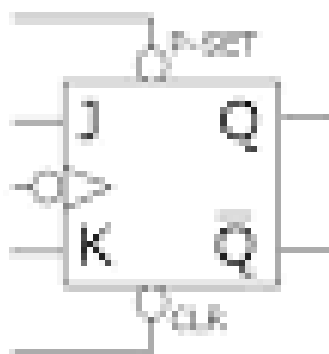
given in Table 8.13.

Table 8.13. Truth table for S-R FF with synchronous preset and clear.

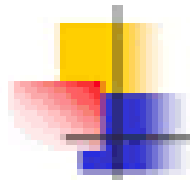
Inputs					Outputs	
P_r	C_r	CLK	S	R	Q_{n+1}	$\bar{Q}_{n+1}$
x	x	0	x	x	NC	NC
0	0	1	x	x	RACE	RACE
0	1	1	x	x	0	1
1	0	1	x	x	1	0
1	1	1	0	0	NC	NC
1	1	1	0	1	0	1
1	1	1	1	0	1	0
1	1	1	1	1	RACE	RACE



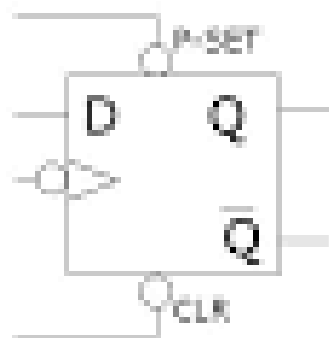
J-K Flip Flop with Preset & Clear



P-SET	CLR	J	K	CLK	Q_{n+1}
0	1	X	X	X	1 (preset)
1	0	X	X	X	0 (clear)
0	0	X	X	X	? (illegal)
1	1	0	0	$\downarrow$	Q_n
1	1	0	1	$\downarrow$	0
1	1	1	0	$\downarrow$	1
1	1	1	1	$\downarrow$	$\bar{Q}_n$



D Flip-Flop w/ Preset & Clear



P-SET	CLR	D	CLK	Q_{n+1}
0	1	X	X	1 (preset)
1	0	X	X	0 (clear)
0	0	X	X	? (illegal)
1	1	0	↓	0
1	1	1	↓	1



TRUTH TABLE, CHARACTERISTIC TABLE AND EXCITATION TABLE OF FFs

S-R F-F

SR Latch

T.T. of SR Latch.

S*	R*	Q
0	0	Not used
0	1	1
1	0	0
1	1	Memory (previous)

$S^* = (S \cdot CLK) = \overline{S} + \overline{CLK}$
 $R^* = (R \cdot CLK) = \overline{R} + \overline{CLK}$

Truth Table

CLK	S	R	Next state
0	x	x	Memory Q_n
1	0	0	Memory Q_n
1	0	1	0
1	1	0	1
1	1	1	Not used.

with $CLK=0$
 $S^* = R^* = 1$
 with $CLK=1$
 $S^* = S = 1$
 $R^* = R = 1$
 with $CLK=1, S=0, R=1$
 $S^* = 1, R^* = 0$

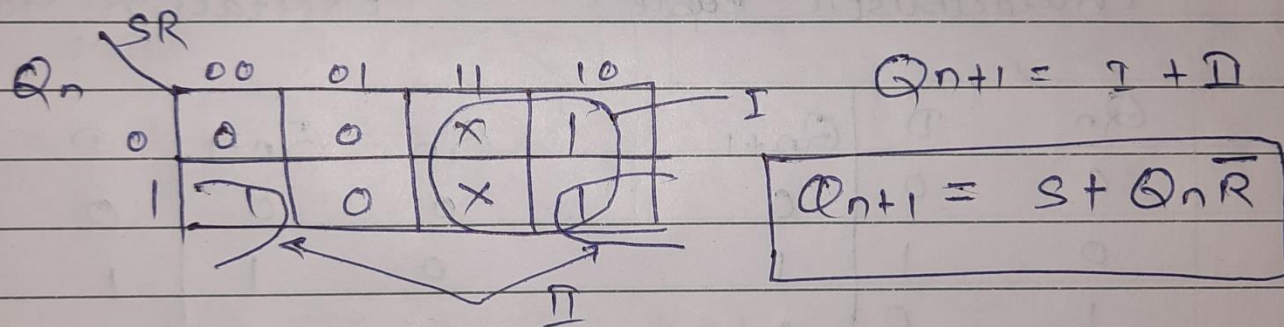
Characteristic Table (Assuming $CLK=1$)

Q_n	S	R	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	x
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	x

SR FF excitation table (w.r.t. characteristic table)

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

To find Q_{n+1} from characteristic table plot K map



D Flip-flop (Data FF)
 T.T. for S-R FF

CLK	S	R	Q_{n+1}		D	S	Q_n
0	x	x	Q_n	memory		R	$\bar{Q}_n$
1	0	0	Q_n				
1	0	1	0				
1	1	0	1				
1	1	1	invalid				

Truth table for D-FF

CLK	D	Q_{n+1}
0	x	Q_n
1	0	0
1	1	1

Characteristic Table

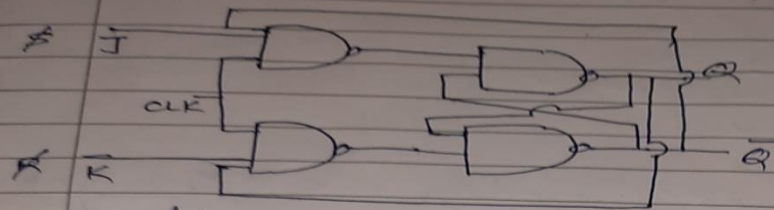
Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Table

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

$Q_{n+1} = D$

J-K FF
 Disadv. of SR FF: With $S=R=1 \rightarrow$ Not used



Truth Table

CLK	J	K	Q_{n+1}
0	x	x	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	$\bar{Q}_n$ (Toggle)

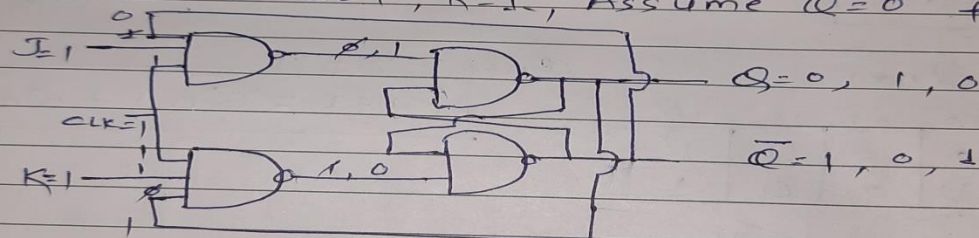
Working

CLK = 0 Q_n

CLK = 1 $J=1, K=0, Q=1, \bar{Q}=0$

CLK = 1 $J=0, K=1, Q=0, \bar{Q}=1$

CLK = 1 $J=1, K=1$, Assume $Q=0 \neq \bar{Q}=1$



$\therefore$ with CLK = 1, $J=1, K=1, Q_{n+1} = \bar{Q}_n$ (Toggling)
characteristic Table of JK FF

Q_n	J	K	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Excitation table (from charact. table)

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

K map for finding J & K, & Q_{n+1}

for J

Q_n	Q_{n+1}
0	0
0	1
1	0
1	1

$$J = Q_{n+1}$$

K map for K (1P)

Q_n	Q_{n+1}
0	0
0	1
1	0
1	1

$$K = \overline{Q_{n+1}}$$

for Q_{n+1}

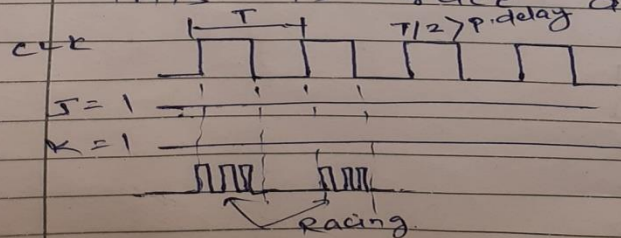
Q_n	JK	00	01	11	10
0	0	0	0	1	1
1	1	1	0	0	1

$$Q_{n+1} = J + \overline{K}$$

$$Q_{n+1} = Q_n J + \overline{Q_n} \cdot \overline{K}$$

Race around condition in J-K FF
 when $clk = 1$, $J = K = 1$ $Q_{n+1} = \overline{Q_n}$
 Assuming $Q = 1$ & $\overline{Q} = 0$
 o/p's will change to $Q = 1, 0, 1, \dots$
 & $\overline{Q} = 0, 1, 0, \dots$

This is race around condition.



Conditions to overcome racing :-

- $T/2 < \text{prop. delay of FF}$
- Edge triggering
- Master-slave FF.

EXCITATION TABLE OF FFS

- The table containing all the output situations and the corresponding input conditions is called as Excitation table of FF

Flip-flop Excitation Tables

- *Excitation tables*: given the required transition from present state to next state, determine the flip-flop input(s).

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

JK Flip-flop

Q	Q^+	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

SR Flip-flop

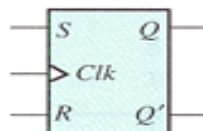
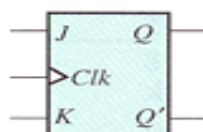
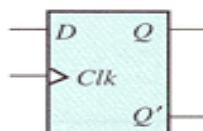
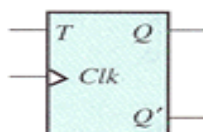
Q	Q^+	D
0	0	0
0	1	1
1	0	0
1	1	1

D Flip-flop

Q	Q^+	T
0	0	0
0	1	1
1	0	1
1	1	0

T Flip-flop

SUMMARY OF FFS

FLIP-FLOP NAME	FLIP-FLOP SYMBOL	CHARACTERISTIC TABLE	CHARACTERISTIC EQUATION	EXCITATION TABLE																																			
SR		<table><tr><th>S</th><th>R</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>NA</td></tr></table>	S	R	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q(next) = S + R'Q$ $SR = 0$	<table><tr><th>Q</th><th>Q(next)</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q(next)	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table><tr><th>J</th><th>K</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Q'</td></tr></table>	J	K	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q(next) = JQ' + K'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>X</td></tr><tr><td>1</td><td>0</td><td>X</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q(next)	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				
D		<table><tr><th>D</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q(next)	0	0	1	1	$Q(next) = D$	<table><tr><th>Q</th><th>Q(next)</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q(next)	D	0	0	0	0	1	1	1	0	0	1	1	1														
D	Q(next)																																						
0	0																																						
1	1																																						
Q	Q(next)	D																																					
0	0	0																																					
0	1	1																																					
1	0	0																																					
1	1	1																																					
T		<table><tr><th>T</th><th>Q(next)</th></tr><tr><td>0</td><td>Q</td></tr><tr><td>1</td><td>Q'</td></tr></table>	T	Q(next)	0	Q	1	Q'	$Q(next) = TQ' + T'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q(next)	T	0	0	0	0	1	1	1	0	1	1	1	0														
T	Q(next)																																						
0	Q																																						
1	Q'																																						
Q	Q(next)	T																																					
0	0	0																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					



CONVERSION OF FFS FROM ONE TYPE TO OTHER TYPE

- SR-FF to JK-FF Conversion
- JK-FF to SR-FF Conversion
- SR-FF to D-FF Conversion
- D-FF to SR-FF Conversion
- JK-FF to T-FF Conversion
- JK-FF to D-FF Conversion
- D-FF to JK-FF Conversion



FF CONVERSIONS STEPS

Steps required to follow in FF conversion:

- Step1: Identify available and required FF
- Step2: Make characteristic table for required FF
- Step3: Make Excitation table for available FF
- Step 4: Write Boolean expression for available FF
- Step5: Draw circuit for realization/Implementation



① JK FF to D FF conversion.

step 1

available FF = JK FF, required FF = D FF

step 2

For D FF char table

Q_n	D	Q_{n+1}	J	K
0	0	0	0	X
0	1	1	1	X
1	0	0	X	1
1	1	1	X	0

step 3 Make excitaⁿ table for avail FF. i.e. J-K FF.

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

step 4 Make K map for J-K FF (available FF)

For J

For K

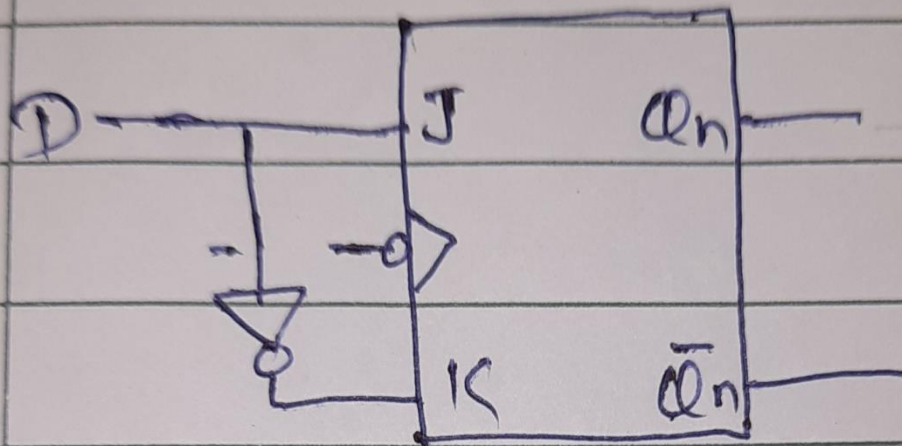
Q_n	D	0	1
0	0	0	1
1	1	X	X

$$\therefore J = D$$

Q_n	D	0	1
0	0	X	X
1	1	1	0

$$K = \bar{D}$$

steps Draw ckt dia.



SR FF to JK FF

Step 1 - Available FF = SR, required FF = JK
 Step 2 - characteristic table for req FF (JK)

Q_n	J	K	Q_{n+1}	S	R
0	0	0	0	0	X
0	0	1	0	0	X
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	X	0
1	0	1	0	0	1
1	1	0	1	X	0
1	1	1	0	0	1

Step 3 - Make excitation table for available FF

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Step 4 - Determine S & R from K map

JK for S

$Q_n \backslash JK$	00	01	11	10
0	0	0	1	1
1	X	0	0	X

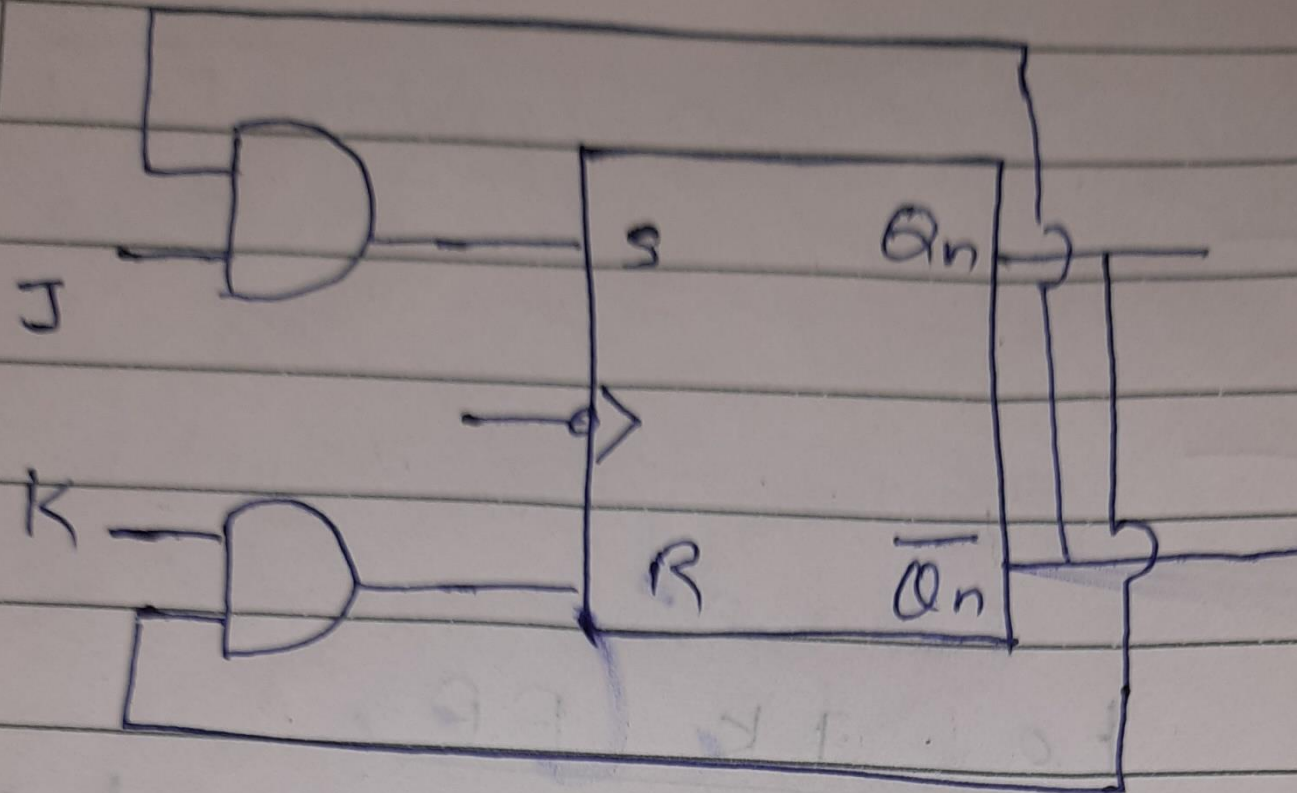
$$S = \overline{Q_n} J$$

JK for R

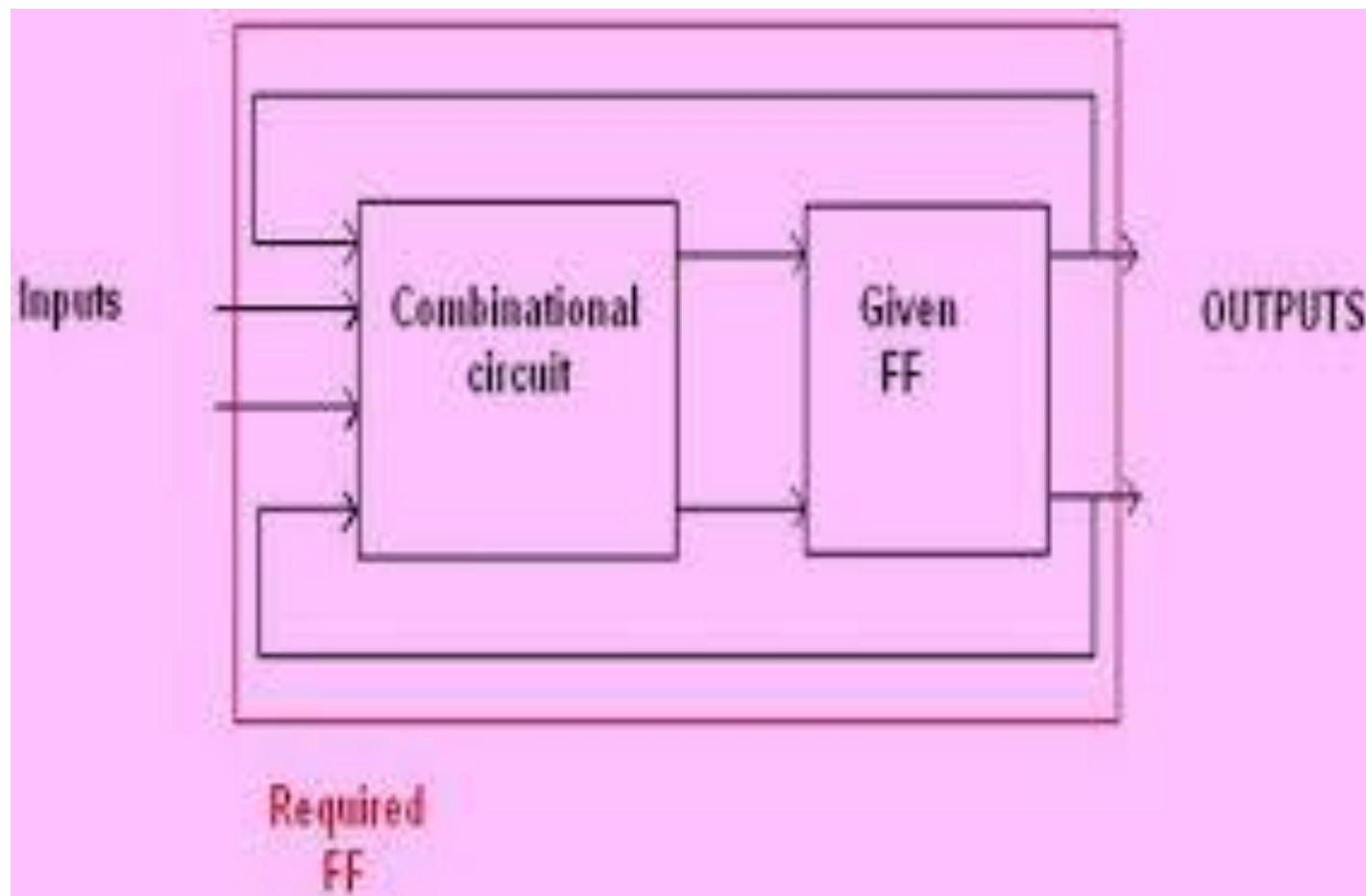
$Q_n \backslash JK$	00	01	11	10
0	X	X	0	0
1	0	1	1	0

$$R = Q_n K$$

Step 5 Implementation



GENERAL MODEL FOR CONVERSION OF FFS



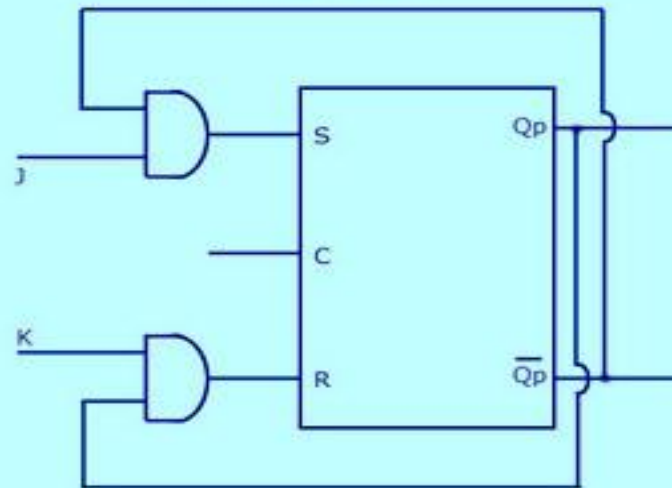
SR TO JK CONVERSION

Conversion Table

J-K Inputs		Outputs		S-R Inputs	
J	K	Qp	Qp+1	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

SR Flip Flop to JK Flip Flop

Logic Diagram



J \ KQp				
	00	01	11	10
0	0 ⁰	X ¹	0 ³	0 ²
1	1 ⁴	X ⁵	0 ⁷	1 ⁶

$$S = \bar{J}Q_p$$

J \ KQp				
	00	01	11	10
0	X ⁰	0 ¹	1 ³	X ²
1	0 ⁴	0 ⁵	1 ⁷	0 ⁶

$$R = KQ_p$$

K-Map

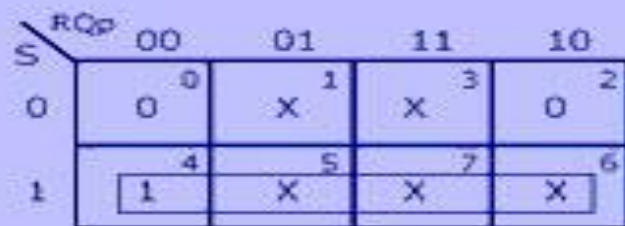
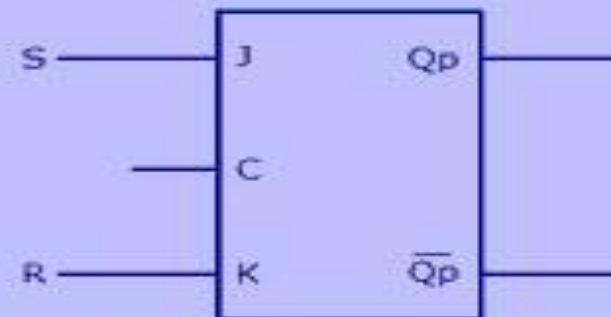


JK TO SR CONVERSION

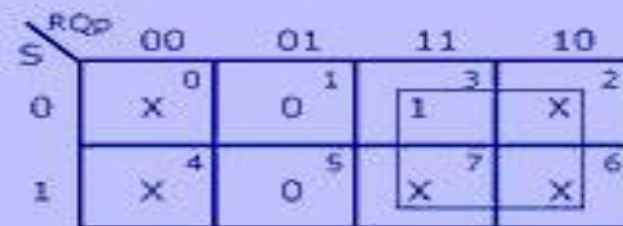
Conversion Table

S-R Inputs		Outputs		J-K Inputs	
S	R	Qp	Qp+1	J	K
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	X	1
1	0	0	1	1	X
1	0	1	1	X	0
1	1	Invalid		Dont care	
1	1	Invalid		Dont care	

Logic Diagram



$$J=S$$



K-maps

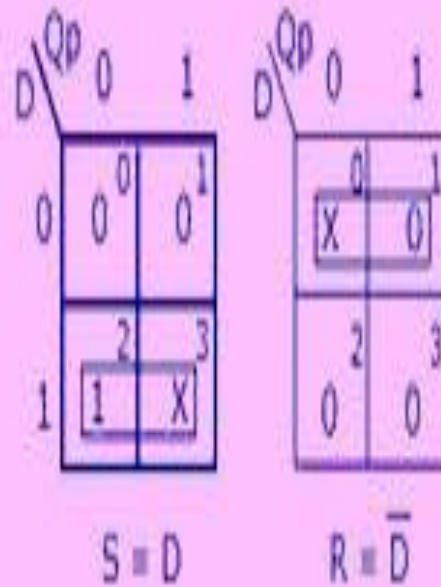
$$K=R$$

SR TO D FF

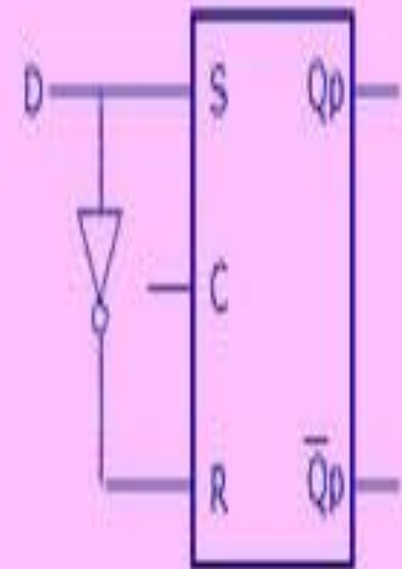
Conversion Table

D Input	Outputs		S-R Inputs	
	Q_p	Q_{p+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

K-maps



Logic Diagram



D TO SR FF

Conversion Table

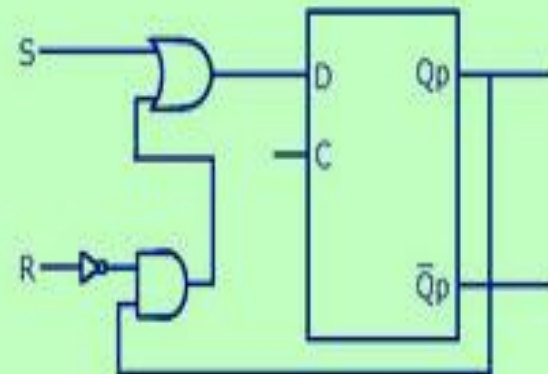
S-R Inputs		Outputs		D Input
S	R	Qp	Qp+1	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	Invalid		Dont care
1	1	Invalid		Dont care

K-map

RQp		00	01	11	10
S	0	0	1	0	0
	1	1	1	X	X

$$D = S + \bar{R}Q_n$$

Logic Diagram

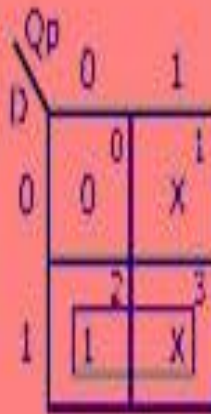


JK TO D FF

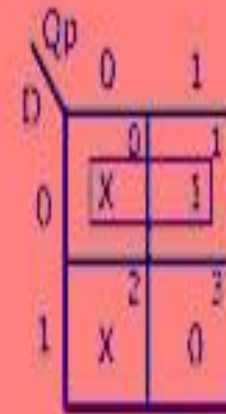
Conversion Table

D Input	Outputs		J-K Inputs	
	Q_p	Q_{p+1}	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	0	X	0

K-maps

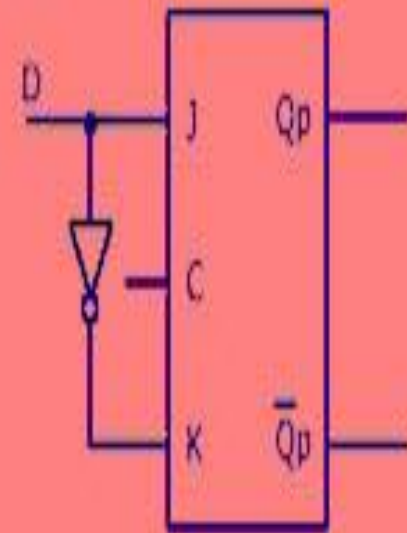


$$J = D$$



$$K = \bar{D}$$

Logic Diagram



D TO JK FF

Conversion Table

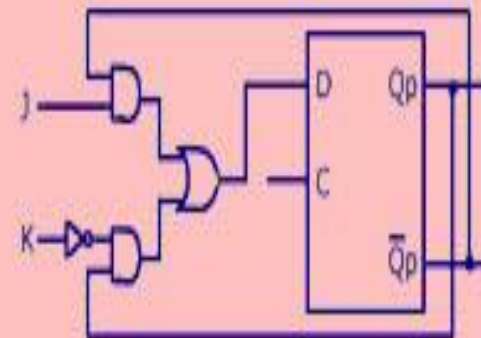
J-K Input		Outputs		D Input
J	K	Q _p	Q _{p+1}	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

K-map

		KQ _p			
		00	01	11	10
J	0	0	1	0	0
	1	0	1	0	0
	4	5	7	6	
	1	1	0	1	

$$D = J\bar{Q}_p + \bar{K}Q_p$$

Logic Diagram



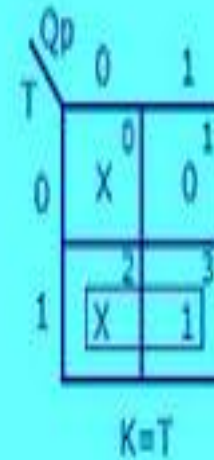
JK TO T FF

J-K Flip Flop to T Flip Flop

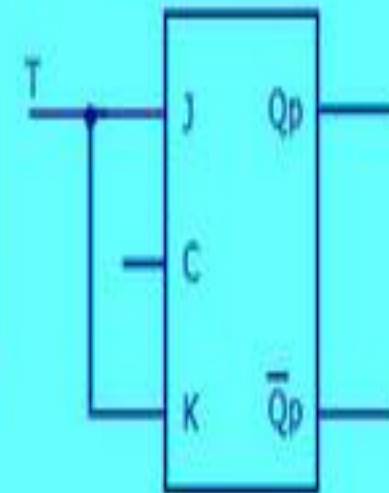
Conversion Table

T Input	Outputs		J-K Inputs	
	Q_p	Q_{p+1}	J	K
0	0	0	0	X
0	1	1	X	0
1	0	1	1	X
1	1	0	X	1

K-maps



Logic Diagram



SUMMARY OF EXCITATION TABLES OF FFS

- Get the **simplified expressions** for each excitation input. If necessary, use Kmaps for simplifying.
- Draw the **circuit diagram** of desired flip-flop according to the simplified expressions using given flip-flop and necessary logic gates.

Present State	Next State	SR flip-flop inputs		D flip-flop input	JK flip-flop inputs		T flip-flop input
Q_t	Q_{t+1}	S	R	D	J	K	T
0	0	0	x	0	0	x	0
0	1	1	0	1	1	x	1
1	0	0	1	0	x	1	1
1	1	x	0	1	x	0	0



APPLICATIONS OF FFS

- As a memory element
- In various types of registers
- In Counters
- In timers
- As a delay element

